SAVEETHA SCHOOL OF ENGINEERING

C PROGRAMMING ASSIGNMENT

Comprehensive System Design and Implementation Challenge

Student Name	BAVADARANI S
Register Number	1912260562
Course Code & Name	CSA0209 C PROGRAMMING
Slot	B
Assignment / Task Title	Comprehensive System Design and Implementation Challenge – District Blood Bank Network
GitHub Repository URL	https://github.com/bavadarini28 / CSA0209-1912260562-BAVADARANI.S.git
Submission Date	13 NOVEMBER 2026

Overview of this Submission

This report answers every task of the **District Blood Bank Network** challenge (50 marks). Part A contains the design work (diagrams, prototypes, pseudocode and analysis). Part B contains the C programs, the measured results and the engineering decisions. Every program in this report was compiled with gcc and run; the output blocks are the real output of those runs.

Part / Task	Marks	What is inside
A1 Decompose into functions	7	Function list, structure chart, prototypes with pass-by-value / const pointer justification, two ways to return two values, global-variable alternative
A2 Recursion	5	Recursive digital root and power, call-stack trace, recurrences, recursion risks, loop / formula alternative
A3 Pointers	5	p+3, *(p+3), p[3], *p++ with memory diagram, pointer-returning function, command-line design, index alternative
A4 Dynamic memory	4	Doubling donor list with realloc failure handling, three memory errors, fixed-growth alternative
A5 File storage	4	Text donor file, binary stock file, in-place update pseudocode, text-rewrite alternative
B1 Recursive functions	7	Tests, call counts, stack-overflow limit, completed code
B2 Adversarial inputs	5	Overflow with scanf / fgets, perror on file errors, wrong arguments, valgrind / ASan
B3 Text vs binary	8	Correctness, size and time at three scales, single-record update, completed code
B4 Final decision	5	Nightly report, memory check, production decision, completed code

PART A – DESIGN & PSEUDOCODE (25 Marks)

Part A needs diagrams and pseudocode only, so **no screenshots are needed in Part A**. The diagrams are already drawn and inserted.

TASK A1 — Decompose the Program into Functions (7 Marks)

Task / Problem Statement

Break the blood bank program (register donors, check eligibility, find eligible donors, update stock after every donation and issue, keep data in files) into functions, design their interfaces, and design a function that returns two values.

Aim

To divide the program into small single-purpose functions with clean interfaces so that each can be tested and reused separately.

A1.1 – Identify the Functions (2 marks)

Function	Purpose (one job each)	Calls
main	Reads argc/argv, validates the command, calls the right function, frees memory	all command functions
register_donor	Validates and adds a new donor, then saves	valid_id, group_index, add_donor, save_donors
valid_id	Checks the check digit of a donor ID	digital_root
digital_root / digit_sum	Recursive digital root (check digit)	each other / itself
is_eligible	True if age 18–65 and at least 90 days since last donation	days_from_date
find_eligible (find_donors)	Counts and lists eligible donors of one group; finds the youngest	is_eligible
load_donors / save_donors	Read / write the text donor file	add_donor, fopen/fgets/fprintf
add_donor	Appends to the growable list, doubling with realloc	realloc
load_stock / update_units / change_stock	Read the binary stock file / update one record in place	fseek, fread, fwrite
lowest_stock	Returns a pointer to the blood group with least stock	–
group_index	Converts "O+" into 0..7	strcmp
days_from_date / date_from_days	Date text ↔ day number	timegm / gmtime
free_list	Releases the donor list	free

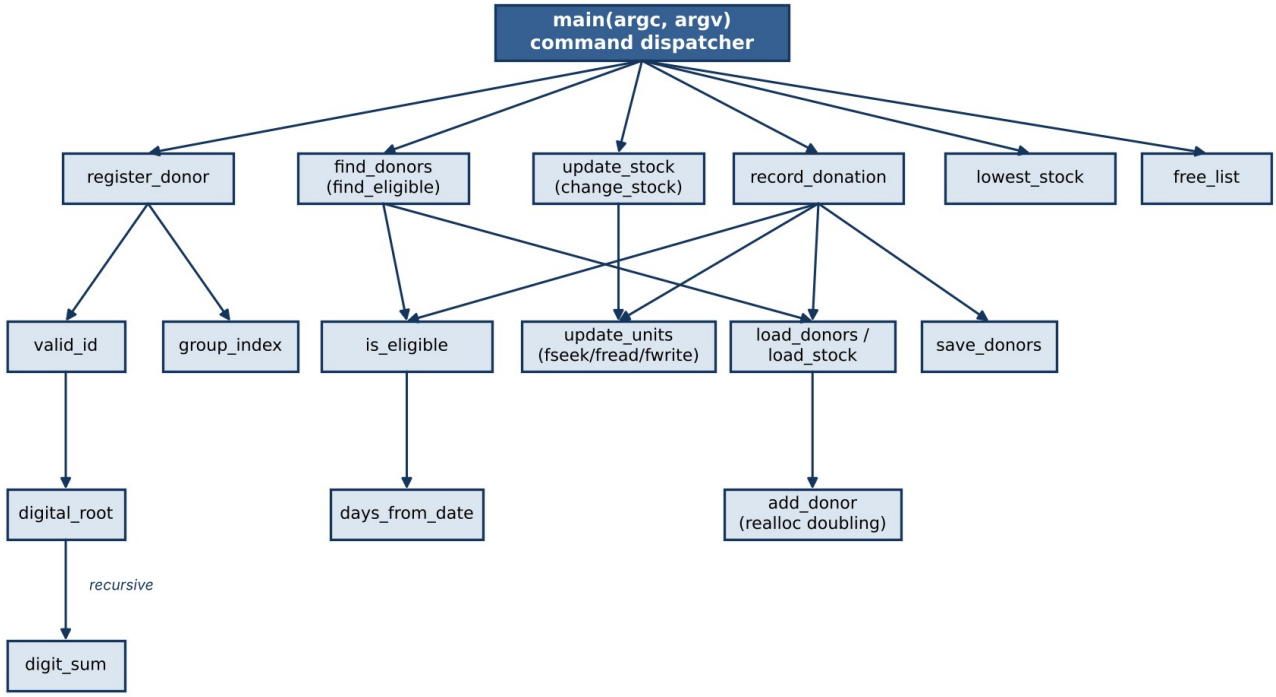


Figure A1 – Structure chart (arrow = "calls").

A1.2 – Design the Interfaces (3 marks)

Rules used for every decision: (1) **small scalars (int, long) are passed by value** – copying is cheap and the caller's variable is protected; (2) **arrays, strings and structures are passed by pointer** – C cannot pass arrays by value, and copying a 72-byte Donor for every call is wasteful; (3) a pointer that is **only read is const** – the compiler then rejects accidental writes and the reader knows the function is harmless; (4) a pointer the function **must change is non-const**.

Prototype	Parameter decisions and justification
int group_index(const char *g)	g by pointer (a string is an array) and const – only compared, never changed.
int digit_sum(long n) int digital_root(long n) int valid_id(int id)	Single numbers, by value – the function works on its own copy, so the caller's ID is never damaged. Return value carries the result.
long days_from_date(const char *s)	s read-only string → const char *. Returns −1 if the text is not a date.
void date_from_days(long d, char out[11])	d by value; out by (non-const) pointer because the function **writes** the text into the caller's buffer.
int is_eligible(const Donor *d, long today)	d by const pointer: avoids copying 72 bytes and guarantees the record is not modified. today by value.
int add_donor(DonorList *list, const Donor *d)	list by non-const pointer: size, capacity and the data pointer all change. d is only copied from → const.
void free_list(DonorList *l)	Non-const: sets data to NULL and size to 0 after freeing (prevents dangling pointer / double free).
int load_donors(const char *path, DonorList *list)	path const string; list is an **output** parameter so non-const.
int save_donors(const char *path, const DonorList *list)	Saving only reads the list → const DonorList *.

Prototype	Parameter decisions and justification
int load_stock(const char *path, Stock *s)	s is the caller's array of 8 records that the function fills → non-const pointer.
int update_units(FILE *fp, int index, int units)	FILE * is a handle that must be shared with the caller (file position changes). index, units by value.
int change_stock(const char *path, int index, int new_units)	Opens, calls update_units, closes. All inputs are read-only values.
void find_eligible(const DonorList *l, const char *group, long today, int *count, int *youngest_id, int print)	List and group are read-only → const. count and youngest_id are **output** pointers (two results). today, print by value.
Stock *lowest_stock(Stock *s, int n)	Non-const Stock * in and out, because the caller will use the returned pointer to **modify** the element. n by value.
int main(int argc, char *argv[])	Standard: argc count by value; argv array of strings supplied by the system.

A1.3 – Return More Than One Value (2 marks)

Alternative 1 – pointer (output) parameters (the version used in the program):

```
ALGORITHM FindEligible(list, group, today, count*, youngestId*)
  INPUT  : list (read-only), group, today
  OUTPUT : *count = number of eligible donors,
           *youngestId = ID of the youngest eligible donor (-1 if none)
BEGIN
  *count <- 0 ; *youngestId <- -1 ; youngestAge <- 1000
  FOR i <- 0 TO list.size - 1 DO
    d <- list.data[i]
    IF d.group = group AND IsEligible(d, today) THEN
      *count <- *count + 1
      IF d.age < youngestAge THEN
        youngestAge <- d.age ; *youngestId <- d.id
      END IF
    END IF
  END FOR
END
```

```
void find_eligible(const DonorList *l, const char *group, long today, int *count, int *youngest_id);
/* call */   int n, id;   find_eligible(&list, "O+", today, &n, &id);
```

Alternative 2 – return a structure:

```
typedef struct { int count; int youngest_id; } EligibleResult;
EligibleResult find_eligible_s(const DonorList *l, const char *group, long today);
/* call */   EligibleResult r = find_eligible_s(&list, "O+", today);   /* r.count, r.youngest_id */
```

Point	Pointer parameters	Return a structure
Call syntax	Caller must declare variables and pass &n, &id	One call, one result variable – reads like a normal function
Cost	Two stores through pointers; nothing copied	8-byte struct is returned in registers by most compilers – equally cheap; a very large struct would be copied
Safety	Caller can pass NULL or forget to read both outputs	Fields are named, no NULL to check
Return value free for errors	Yes – function can return 0/1 status as well	Status must become another field

Extending later (add a 3rd result)	Prototype changes for every caller	Add a field; old calls still compile
Best used when	Many results, large results, or status code needed	A small fixed bundle of related results

Conclusion: for exactly two small related values the structure is cleaner and safer; the pointer version is the classic C idiom, keeps the return value for a status code and avoids copying when results are big. Both run in O(n) time and O(1) extra space.

Alternative Concept and Complexity Analysis – shared global variables

What the solution would be. Declare DonorList donors; Stock stock[8]; int eligible_count; int youngest_id; at file level. Then find_eligible() takes no list argument, reads donors directly and writes eligible_count and youngest_id; add_donor(id, group) modifies donors, and so on.

Aspect	Parameters / pointers	Global variables
Running time	O(n) for find_eligible	Still O(n). Reading a global is as fast as reading through a pointer (a few cycles either way).
Memory	List lives where the caller puts it; stack holds a few pointers	Same data, same size. Globals live for the whole program (static storage), so the memory can never be released early. Saves only a few bytes of parameter passing.
Coupling	Low – a function depends only on what is in its prototype	High (“common coupling”). Every function silently depends on the same hidden variables; a change in one place can break another.
Testing	Build a small list in the test, call, compare	Test must first set up and afterwards reset the global state; tests influence each other and depend on the order they run in.
Reuse	Same function works on any list (e.g. one list per hospital)	Works on exactly one list. A second list or a second thread needs the source changed. Not thread-safe.
Readability	Prototype documents inputs and outputs	Reader must read the whole body to learn what it reads and changes.

Conclusion: the time and memory are essentially unchanged, but the engineering quality is worse. Parameters and pointers are therefore used in this design; the only global in the code is the instrumentation counter calls in B1, which is not part of the blood bank logic.

Result

The program is divided into 13 small functions with a clear call hierarchy. Every parameter is passed by value or by (const) pointer for a stated reason, and two designs for returning two values were compared.

TASK A2 — Design with Recursion (5 Marks)

Task / Problem Statement

Donor IDs carry a check digit equal to the digital root of the other digits. The blood bank also computes base^n for a reporting formula. Design both recursively.

Aim

To express the digital root and fast exponentiation as recursive algorithms, trace them and analyse their cost and risks.

A2.1 – Recursive Design (2 marks)

```
ALGORITHM DigitSum(n)                ALGORITHM DigitalRoot(n)
  INPUT  : n >= 0                      INPUT  : n >= 0
  OUTPUT : sum of the digits of n      OUTPUT : single digit 0..9
BEGIN                                BEGIN
  IF n < 10 THEN                       IF n < 10 THEN
    RETURN n            (base)           RETURN n            (base)
  END IF                               END IF
  RETURN (n MOD 10) + DigitSum(n DIV 10)  RETURN DigitalRoot(DigitSum(n))   (recursive)
END                                  END
```

```
ALGORITHM Power(base, n)             ALGORITHM PowerLinear(base, n)
  INPUT  : base, n >= 0                INPUT  : base, n >= 0
  OUTPUT : base^n                      OUTPUT : base^n
BEGIN                                BEGIN
  IF n = 0 THEN                        IF n = 0 THEN
    RETURN 1            (base)           RETURN 1            (base)
  END IF                               END IF
  half <- Power(base, n DIV 2)  (ONCE)    RETURN base * PowerLinear(base, n - 1)
  IF n MOD 2 = 0 THEN                  END
    RETURN half * half
  ELSE
    RETURN half * half * base
  END IF
END
```

Function	Base case	Recursive case
DigitalRoot(n)	n < 10 → return n	DigitalRoot(DigitSum(n)) – the number gets smaller every time
DigitSum(n)	n < 10 → return n	(n mod 10) + DigitSum(n div 10) – one digit less each call
Power(base, n) [halving]	n = 0 → return 1	(Power(base, n div 2))², times base if n is odd – exponent halves
PowerLinear(base, n)	n = 0 → return 1	base × PowerLinear(base, n−1) – exponent drops by 1

A2.2 – Trace the Calls (2 marks)

Power(2, 10): Power(10) → Power(5) → Power(2) → Power(1) → Power(0) = 1. Returning: Power(1) = 1·1·2 = 2; Power(2) = 2^2 = 4; Power(5) = 4^2·2 = 32; Power(10) = 32^2 = **1024**. Five calls, depth 5.

DigitalRoot(9875): DigitSum(9875) = 5 + DigitSum(987) = 5 + 7 + DigitSum(98) = 5+7+8+DigitSum(9) = 5+7+8+9 = 29, so DigitalRoot(29) → DigitSum(29) = 11 → DigitalRoot(11) → DigitSum(11) = 2 → DigitalRoot(2) = **2** (check digit 2).

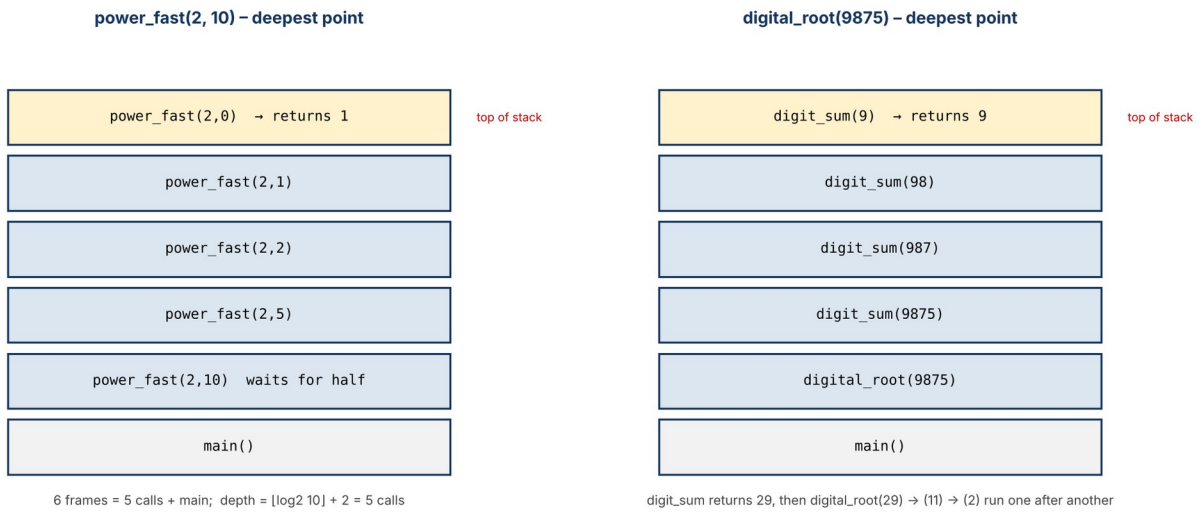


Figure A2 – Call stack at the deepest point of each trace (top of stack is the newest call).

Recurrences (T = number of calls):

Version	Recurrence	Solution	Value for n = 30
Linear	$T(0) = 1;\ T(n) = 1 + T(n - 1)$	$T(n) = n + 1 \rightarrow O(n)$	31
Halving (one recursive call)	$T(0) = 1;\ T(n) = 1 + T(\lfloor n/2 \rfloor)$	$T(n) = \lfloor \log_2 n \rfloor + 2 \rightarrow O(\log n)$	6
Halving but two calls (the wrong way)	$T(0) = 1;\ T(n) = 1 + 2 \cdot T(\lfloor n/2 \rfloor)$	$T(n) = 2^{(\lfloor \log_2 n \rfloor + 2)} - 1 \rightarrow O(n)$	63
DigitSum with d digits	$T(1 \text{ digit}) = 1;\ T(d) = 1 + T(d - 1)$	$T(d) = d$	–

A2.3 – Recursion Risks (1 mark)

- **Missing base case:** the function never stops calling itself. Every call adds a stack frame (return address, parameters, locals), the stack (about 8 MB) fills up and the program crashes with a **stack overflow** (segmentation fault). B1.4 shows this happening at n = 261,402.
- **Replace recursion with a loop when** the depth grows with the input size (linear recursion such as PowerLinear, or a list with 50,000 donors); when a loop is just as clear; when the function is called very often (call overhead); or on small-memory systems. Keep recursion when the depth is small and the problem is naturally recursive (halving, trees).

Alternative Concept and Complexity Analysis – loop or formula for the digital root

```
ALGORITHM DigitalRootLoop(n)            ALGORITHM DigitalRootFormula(n)
BEGIN                                   BEGIN
  WHILE n >= 10 DO                        IF n = 0 THEN RETURN 0
    s <- 0                              RETURN 1 + (n - 1) MOD 9
    WHILE n > 0 DO                      END
      s <- s + n MOD 10 ; n <- n DIV 10
    END WHILE
    n <- s
  END WHILE
  RETURN n
END
```

Version	Time	Extra stack space	Comment
Recursive (DigitSum depth d)	O(d), d = number of digits	O(d) frames	Clear, mirrors the definition

Loop	O(d) for the first pass; later passes work on at most 2-digit numbers	O(1)	No stack growth; works for any size
Formula 1 + (n−1) mod 9	**O(1)** – one division	O(1)	Fastest, but needs the maths insight; special-case n = 0

For a 64-bit number $d \le 19$, so all three are practically instant; the choice is about clarity and safety. B1 confirms that the formula and the recursive function agree on every test value.

Result

Recursive designs with base cases, traces, recurrences ($n + 1$, $\lfloor \log_2 n \rfloor + 2$, $\sim 2n$) and risks were produced; loop and formula alternatives were compared.

TASK A3 — Design with Pointers (5 Marks)

Task / Problem Statement

Explain pointer arithmetic on the eight-element stock array, design a function that returns a pointer to the lowest stock, and design the command-line interface.

Aim

To use pointers to arrays and functions correctly and safely, and to read program input through argc / argv.

A3.1 – Pointer Operations (2 marks)

Given int stock[8]; int *p = stock; (p holds the address of stock[0]; an int is 4 bytes, so p + k is the base address plus 4·k bytes):

Expression	Meaning	Example (base 0x1000)
p + 3	A **pointer** to the 4th element, &stock[3]. Nothing is read; the address moves 3 × sizeof(int) = 12 bytes.	0x100C
*(p + 3)	The **value** stored at that address – the content of stock[3].	value of stock[3]
p[3]	Exactly the same as *(p + 3); the index notation is just shorthand.	value of stock[3]
*p++	++ binds tighter than *: the expression value is *p (that is stock[0]), **then** p is advanced by one element. p now holds &stock[1]. (It is not (*p)++, which would increase the stock value.)	p: 0x1000 → 0x1004

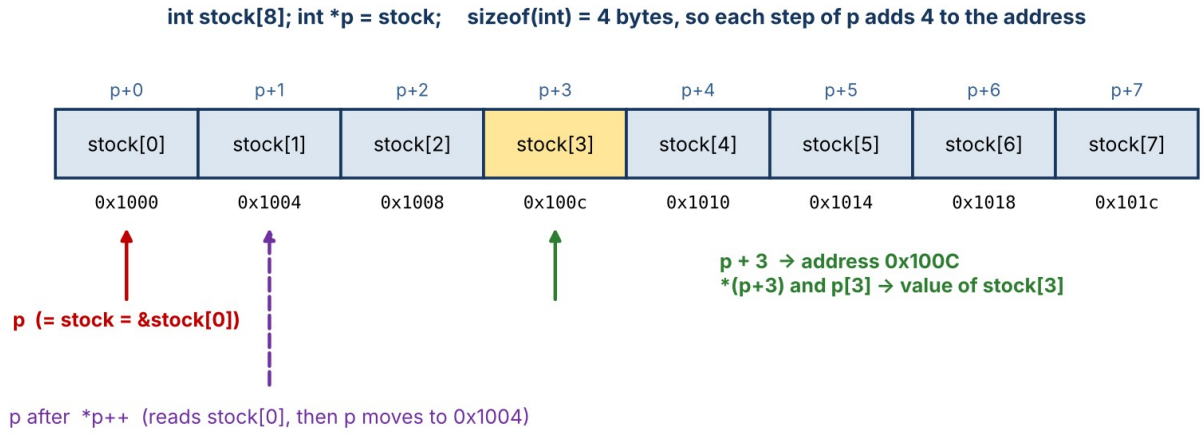


*Figure A3 – Memory diagram of stock[8], p, p + 3 and p after *p++.*

A3.2 – Pointers and Functions (2 marks)

```
ALGORITHM LowestStock(stock*, n)
  INPUT  : stock = address of first element, n > 0 = element count
  OUTPUT : address of the element holding the smallest units
BEGIN
  low <- &stock[0]
  FOR i <- 1 TO n - 1 DO
    IF stock[i].units < (*low).units THEN low <- &stock[i]
  END FOR
  RETURN low
END
```

```
Stock *lowest_stock(Stock *s, int n)   /* pointer INTO the caller's array */
{
   Stock *low = &s[0];
   for (int i = 1; i < n; i++)
      if (s[i].units < low->units) low = &s[i];
```

```
   return low;
}
/* caller:  lowest_stock(stock, 8)->units += 10;   updates the array directly */
```

Why returning a pointer to a local variable is wrong. A local variable lives in the function's stack frame, and that frame is destroyed when the function returns. The returned address then points to memory that will be reused by the next call, so the pointer is a **dangling pointer**: reading it gives garbage, writing corrupts other data (undefined behaviour). The design is safe because the pointer refers to the caller's array, which outlives the call. Other safe sources are malloced memory or a static variable.

A3.3 – Command-Line Interface (1 mark)

Command (argv[1] ...)	argc	Action	Checks
bloodbank stock	2	Print units of all 8 groups	No extra arguments
bloodbank find O+	3	List eligible donors of group O+, count and youngest	group in {A+,A-,B+,B-,AB+,AB-,O+,O-}
bloodbank issue A- 2	4	Reduce stock of A- by 2	valid group; units is a whole number 1..1000 (strtol); enough stock
bloodbank donate 10023	3	Record a donation by donor ID, add 1 unit	donor exists and is eligible
bloodbank register 10089 Vimal O+ 27 9812345670	7	Add a new donor	ID check digit, name < 32 chars, group, age 18–65
bloodbank lowest	2	Show the group with least stock	No extra arguments

Invalid arguments: the program never guesses. It (1) checks argc first, (2) compares argv[1] with the known command names, (3) validates every value, (4) prints a one-line message on **stderr**, plus the usage text for a missing or unknown command, and (5) ends with exit status **1** (0 means success). Numbers are converted with strtol and the end pointer is checked so abc or -5 is rejected.

Alternative Concept and Complexity Analysis – return the index

```
int lowest_index(const Stock *s, int n)
{ int low = 0; for (int i = 1; i < n; i++) if (s[i].units < s[low].units) low = i; return low; }

/* caller */   int i = lowest_index(stock, 8);   stock[i].units += 10;      /* pointer version:  lowest_stock(stock, 8)->units += 10; */
```

	Returns pointer	Returns index
Time	O(n) – one pass	O(n) – one pass
Space	O(1)	O(1)
Calling code	Shortest: lowest_stock(stock,8)->units += 10	Two steps: get index, then stock[i]; the index can also be printed or used to name the group (GROUP_NAMES[i])
Safety if the array is later reallocated	**Unsafe.** realloc may move the block; a stored Stock * still points to the old, freed block → dangling pointer	**Safe.** An index is a position, not an address – stock[i] is looked up again in the new block
Empty array	Can return NULL – caller must test	Needs a special value such as −1

Conclusion: both cost the same; use the pointer for an immediate update, and the index when the pointer must be kept across any operation that can reallocate the array.

Result

Pointer expressions were explained with a memory diagram, a safe pointer-returning function was designed and the command-line interface with its error handling was specified.

TASK A4 — Design Dynamic Memory Use (4 Marks)

Task / Problem Statement

The number of donors is unknown (50 to 50,000 or more). Store them in memory without wasting space and without losing data.

Aim

To design a growable array that doubles with realloc, handle allocation failure, and avoid memory errors.

A4.1 – Growable Donor List (2 marks)

```
typedef struct { Donor *data; int size, capacity; } DonorList;   /* starts as {NULL, 0, 0} */
```

```
ALGORITHM AddDonor(list*, donor)
  INPUT  : list (data, size, capacity), donor record
  OUTPUT : 1 on success, 0 if memory could not be obtained (list unchanged)
BEGIN
  IF list.size = list.capacity THEN                // list is full
    IF list.capacity = 0 THEN newCap <- 2 ELSE newCap <- 2 * list.capacity
    p <- realloc(list.data, newCap * sizeof(Donor))   // temporary pointer
    IF p = NULL THEN
      RETURN 0                                     // old block still valid, no data lost
    END IF
    list.data <- p ; list.capacity <- newCap
  END IF
  list.data[list.size] <- donor
  list.size <- list.size + 1
  RETURN 1
END
```

If realloc() fails it returns NULL and leaves the old block untouched. Because the result is first stored in the temporary p, list->data still points to the old, valid block: the donors already in memory stay usable, the function reports failure (return 0), the caller prints an error and can still save the file and free the list. Writing list->data = realloc(list->data, ...) directly would overwrite the only pointer with NULL, leak the old block and lose all donors.

A4.2 – Memory Errors (2 marks)

Error	Meaning	How it could happen in the blood bank	How the design prevents it
Memory leak	Allocated memory is never freed, so it is lost until the program ends	find loads donors into a list and returns early on an error without calling free; or list->data = realloc(list->data, n) fails and the old block is lost	Single exit path: main always calls free_list(&list); temporary pointer p in add_donor; checked with valgrind (“All heap blocks were freed”)
Dangling pointer	A pointer that still holds the address of memory that was freed or moved	Donor *d = &list.data[0]; then more donors are added; realloc moves the block; d->id now reads freed memory. Also free(list.data) followed by reading list.data[0]	Never keep Donor * across add_donor; use indices. free_list sets data = NULL, size = capacity = 0. lowest_stock points into a fixed array that is never reallocated
Double free	free called twice on the same block → heap corruption / crash	An error path calls free_list(&list) and then main calls it again at the end	free_list sets data = NULL afterwards, and free(NULL) is harmless; one owner (main) for the list

B2 reproduces all three on purpose and shows valgrind’s report for each.

Alternative Concept and Complexity Analysis – grow by a fixed 10 records

The solution would be newCap = capacity + 10. Every 10th insertion triggers a realloc, and (in the worst case, when the block must move) all existing elements are copied.

Donors n	Elements copied: doubling	Elements copied: +10 each time	Unused slots at the end: doubling	Unused: +10
1,000	1,022	49,500	24	≤ 9
10,000	16,382	4,995,000	6,384	≤ 9
100,000	131,070	499,950,000	31,072	≤ 9
1,000,000	1,048,574	49,999,500,000	48,576	≤ 9

(Counts computed by simulating both strategies; they are worst-case counts, because a real realloc can sometimes extend the block in place.)

- **Doubling:** copies $1 + 2 + 4 + \ldots < 2n$ elements in total → **O(n)** overall, O(1) amortised per insertion. Cost: up to almost half of the capacity can be unused (capacity < 2n, so waste < n elements).
- **Fixed +10:** copies $10 + 20 + 30 + \ldots \approx n^2/20$ elements → **$O(n^2)$**. At 100,000 donors that is 500 million element copies against 131 thousand. Waste is only 9 elements at most.

Conclusion: doubling trades a little memory for a huge saving in time, so it is used.

Result

A growable list with safe realloc handling was designed, three memory errors were explained with prevention, and the growth strategies were compared (O(n) against $O(n^2)$).

TASK A5 — Design the File Storage (4 Marks)

Task / Problem Statement

Store donors and stock in files so that nothing is lost when the computer is switched off, and update one stock record without rewriting the file.

Aim

To choose a suitable file format for each kind of data and to update a record in place with fseek.

A5.1 – Text or Binary? (2 marks)

Donor file – text, one donor per line, written with fprintf:

```
10012,Anitha,O+,24,9876543210,2026-05-01
10023,Karthik,O+,35,9876501234,2026-10-20

fprintf(fp, "%d,%s,%s,%d,%s,%s\n", id, name, group, age, phone, last_donation);
```

- Names and phone numbers have **variable length**, so a text line wastes no space.
- The file can be **read and corrected with any editor**, copied between computers, e-mailed and compared with a diff tool; it does not depend on int size, byte order or struct padding.
- Donors are loaded once into memory, so line-by-line reading speed is acceptable (B3: 100,000 records read in about 21 ms).

Stock file – binary, fixed-size records, written with fwrite:

```
typedef struct { int group_code; int units; } Stock;    /* always 8 bytes */
fwrite(stock, sizeof(Stock), 8, fp);      /* record i starts at byte i * sizeof(Stock) */
```

- Every record has the **same size**, so record *i* is at byte offset i × sizeof(Stock) and can be reached directly with fseek (random access).
- Stock changes after **every donation and issue**; a binary file lets one record be changed in O(1) without touching the others.
- No text conversion is needed (fast, exact integers).

A5.2 – Update in Place (2 marks)

```
ALGORITHM UpdateUnits(fp, index, units)
  INPUT  : fp = binary stock file opened "r+b", index = record number, units = new value
  OUTPUT : 1 on success, 0 on failure
BEGIN
  IF fseek(fp, index * sizeof(Stock), SEEK_SET) <> 0 THEN RETURN 0     // go to the record
  IF fread(&s, sizeof(Stock), 1, fp) <> 1 THEN RETURN 0              // read it (fails past end of file)
  s.units <- units                                    // change it in memory
  IF fseek(fp, -sizeof(Stock), SEEK_CUR) <> 0 THEN RETURN 0          // step back one record
  IF fwrite(&s, sizeof(Stock), 1, fp) <> 1 THEN RETURN 0             // write it back
  RETURN 1
END
```

The step back is needed because fread moved the file position to the start of the next record; without it the write would damage the following record. A seek is required whenever a program switches between reading and writing the same file.

Why this is difficult for the text file. Lines have **different lengths**, so the position of line *k* cannot be calculated – the file must be read from the start to find it. If the new text has a different length (e.g. 9 → 10), every byte after it would have to move, which a file cannot do. The only safe method is to rewrite the whole file (usually to a temporary file that then replaces the original).

Alternative Concept and Complexity Analysis – text stock file rewritten after every update

	Binary file, update in place	Text file, rewrite whole file
Time per update (n records)	**O(1)**: one seek, one read, one write	**O(n)**: read and parse n lines, write n lines

	Binary file, update in place	Text file, rewrite whole file
Measured (B3.3, 100,000 donor records)	0.0022 ms	8.14 ms (3,696 × slower)
File size	Exactly n × 8 bytes (8 groups → 64 bytes)	About 6–8 bytes per line (O+,40\n), variable – slightly smaller or similar for small numbers
Crash safety	Interrupted write damages one record only	Interrupted rewrite can truncate the file; needs temp file + rename
Human readable	No	Yes

For only 8 stock records either is fast, but the stock file is updated constantly and the design must stay efficient if the district grows (more groups, more hospitals), so the in-place binary design is chosen.

Result

Text format for donors and binary fixed-size format for stock were justified, and an in-place update algorithm was given with its O(1) versus O(n) comparison.

PART B – IMPLEMENTATION & EXPERIMENTAL INVESTIGATION (25 Marks)

All programs were compiled with gcc (version 13.3) on Linux. **Every box labelled "PASTE SCREENSHOT" must be replaced by a screenshot of the same command run on your own computer.** The text output printed under each box is the output obtained in my run, so you can check that your screenshot matches. Timings and the crash value depend on the computer, so your numbers may differ slightly; if they do, update the tables with your own figures.

How to run (for every program): gcc -Wall -O0 -o name name.c then ./name. Programs that need a file are run from the folder that contains donors.txt and stock.dat.

TASK B1 — Implement and Instrument the Recursive Functions (7 Marks)

Task / Problem Statement

Implement the digital root and both power functions, test them, count the recursive calls for n = 10, 100, 1,000 and 10,000, and find the value of n at which linear recursion crashes.

Aim

To verify the recursive designs of A2 and to show with measurements why halving is better than linear recursion and why recursion depth is limited.

Concept / Theory

Each recursive call creates a stack frame. Linear recursion makes n + 1 calls and needs n frames; halving recursion makes about $\log_2 n + 2$ calls. The stack is limited (about 8 MB), so a deep recursion ends in a stack overflow. The compiler is run with -O0 so that it does not turn the recursion into a loop.

Procedure / Algorithm

1. Write digit_sum, digital_root, power_linear, power_fast and an iterative power_iter (pseudocode of A2.1).
2. Add a global counter calls that every power function increments on entry.
3. Test with ordinary values, 0, 1 and one value of my own choice and compare the three power versions and the formula.
4. For n = 10, 100, 1,000, 10,000 reset the counter, run each version, and print the counts (base = 1 so the result never overflows).
5. Complete TODO 1 of the given program, run it, then change half * half to two calls and count again.
6. Run the linear version with growing n until it crashes (bisection); repeat with the iterative version.

Program / Source Code

Program 1 – b1_recursion.c (tests and call counts):

```
#include <stdio.h>

long long calls = 0;                    /* number of recursive calls made */

/* ---------- digital root ---------- */
int digit_sum(long n)                    /* recursive sum of digits */
{
   if (n < 10) return (int)n;           /* base case: one digit */
   return (int)(n % 10) + digit_sum(n / 10);
}
int digital_root(long n)
{
   if (n < 10) return (int)n;           /* base case: already one digit */
   return digital_root(digit_sum(n));    /* recursive case */
}
int digital_root_formula(long n) { return n == 0 ? 0 : 1 + (int)((n - 1) % 9); }

/* ---------- power ---------- */
long long power_linear(long long base, int exp)
{
```

```
    calls++;
    if (exp == 0) return 1;
    return base * power_linear(base, exp - 1);
}
long long power_fast(long long base, int exp)
{
    calls++;
    if (exp == 0) return 1;                 /* base case */
    long long half = power_fast(base, exp / 2);          /* computed ONCE */
    if (exp % 2 == 0) return half * half;
    return half * half * base;
}
long long power_fast_twice(long long base, int exp)       /* the "wrong" version */
{
    calls++;
    if (exp == 0) return 1;
    long long r = power_fast_twice(base, exp / 2) * power_fast_twice(base, exp / 2);
    return (exp % 2 == 0) ? r : r * base;
}
long long power_iter(long long base, int exp)             /* square-and-multiply */
{
    long long result = 1;
    while (exp > 0) {
        if (exp & 1) result *= base;
        base *= base;
        exp >>= 1;
    }
    return result;
}

int main(void)
{
    puts("== Test table: digital root ==");
    long dr_in[] = {0, 1, 9, 38, 9875, 99999, 493193};
    for (int i = 0; i < 7; i++)
        printf("digital_root(%ld) = %d   formula = %d\n", dr_in[i], digital_root(dr_in[i]), digital_root_formula(dr_in[i]));

    puts("\n== Test table: power ==");
    long long tb[][2] = {{2,10},{5,0},{7,1},{3,13},{0,5},{1,1000}};
    for (int i = 0; i < 6; i++) {
        long long b = tb[i][0]; int e = (int)tb[i][1];
        printf("%lld^%d: linear=%lld fast=%lld iter=%lld\n", b, e, power_linear(b,e), power_fast(b,e), power_iter(b,e));
    }

    puts("\n== Recursive call counts (base = 1 so values never overflow) ==");
    printf("%8s %10s %8s %12s\n", "n", "linear", "fast", "fast-twice");
    int ns[] = {10, 100, 1000, 10000};
    for (int i = 0; i < 4; i++) {
        long long cl, cf, ct;
        calls = 0; power_linear(1, ns[i]);     cl = calls;
        calls = 0; power_fast(1, ns[i]);       cf = calls;
        calls = 0; power_fast_twice(1, ns[i]); ct = calls;
        printf("%8d %10lld %8lld ", ns[i], cl, cf);
        if (ct >= 0) printf("%12lld\n", ct); else printf("%12s\n", "(see note)");
    }
    calls = 0; power_fast_twice(1, 30);
    printf("\nfast-twice for n = 30: %lld calls\n", calls);
    return 0;
}
```

Program 2 – the given program with TODO 1 completed (b1_todo.c):

```
#include <stdio.h>

long long calls = 0; /* number of recursive calls made */

/* Linear recursion: base^exp = base * base^(exp-1) */
long long power_linear(long long base, int exp)
{
   calls++;
   if (exp == 0) return 1;
   return base * power_linear(base, exp - 1);
}

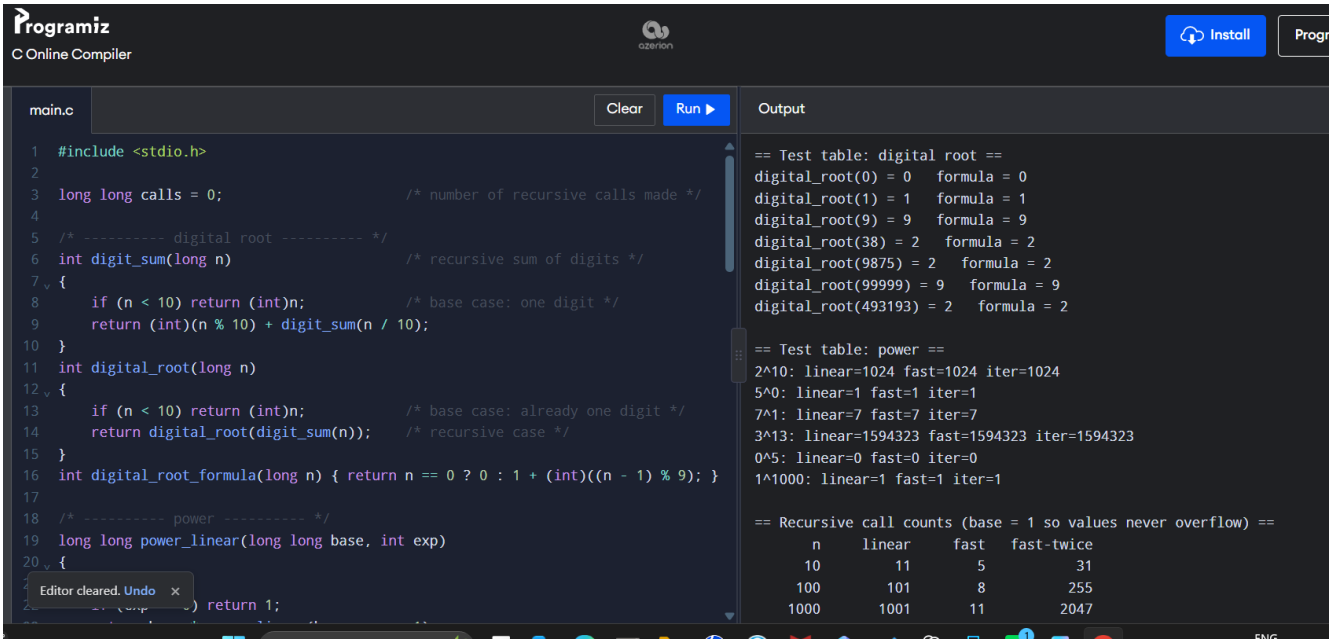
/* Divide-and-conquer recursion: compute base^(exp/2) once and square it. */
long long power_fast(long long base, int exp)
{
   calls++;
   if (exp == 0) return 1;                    /* base case */
   long long half = power_fast(base, exp / 2);  /* computed ONCE */
   if (exp % 2 == 0) return half * half;       /* even exponent */
   return half * half * base;                  /* odd exponent  */
}

int main(void)
{
   calls = 0;
   long long a = power_linear(2, 30);
   printf("linear: %lld in %lld calls\n", a, calls);
   calls = 0;
   long long b = power_fast(2, 30);
   printf("fast : %lld in %lld calls\n", b, calls);
   return 0;
}
```

Program 3 – stack-limit program (b1_limit.c):

```
#include <stdio.h>
#include <stdlib.h>
long long power_linear(long long base, long exp) { if (exp == 0) return 1; return base * power_linear(base, exp - 1); }
long long power_iter(long long base, long exp) { long long r = 1; while (exp-- > 0) r *= base; return r; }
int main(int argc, char **argv)
{
   long n = atol(argv[1]);
   long long v = (argc > 2) ? power_iter(1, n) : power_linear(1, n);
   printf("%s n=%ld ok (result %lld)\n", argc > 2 ? "iterative" : "recursive", n, v);
   return 0;
}
```

Output Screenshot

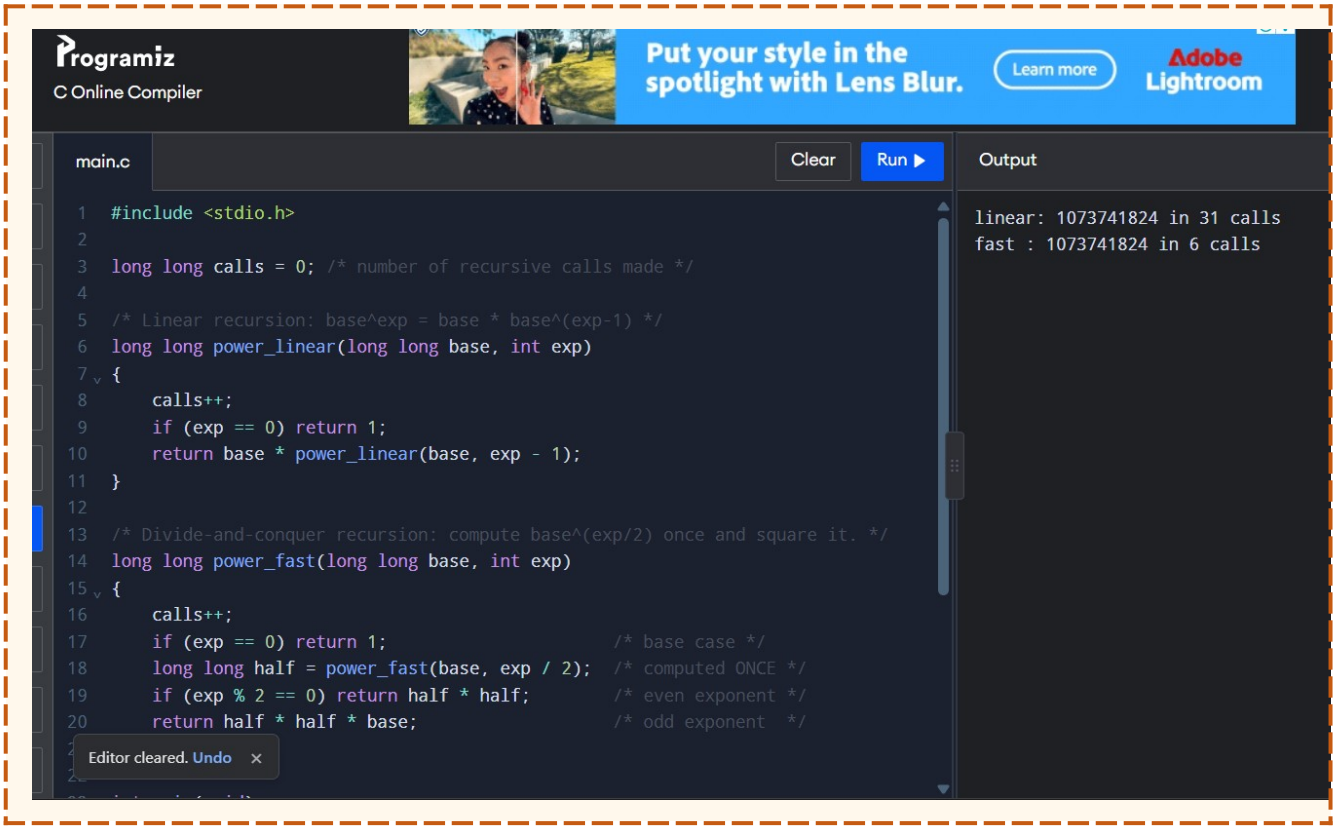



```
== Test table: digital root ==
digital_root(0) = 0   formula = 0
digital_root(1) = 1   formula = 1
digital_root(9) = 9   formula = 9
digital_root(38) = 2   formula = 2
digital_root(9875) = 2   formula = 2
digital_root(99999) = 9   formula = 9
digital_root(493193) = 2   formula = 2

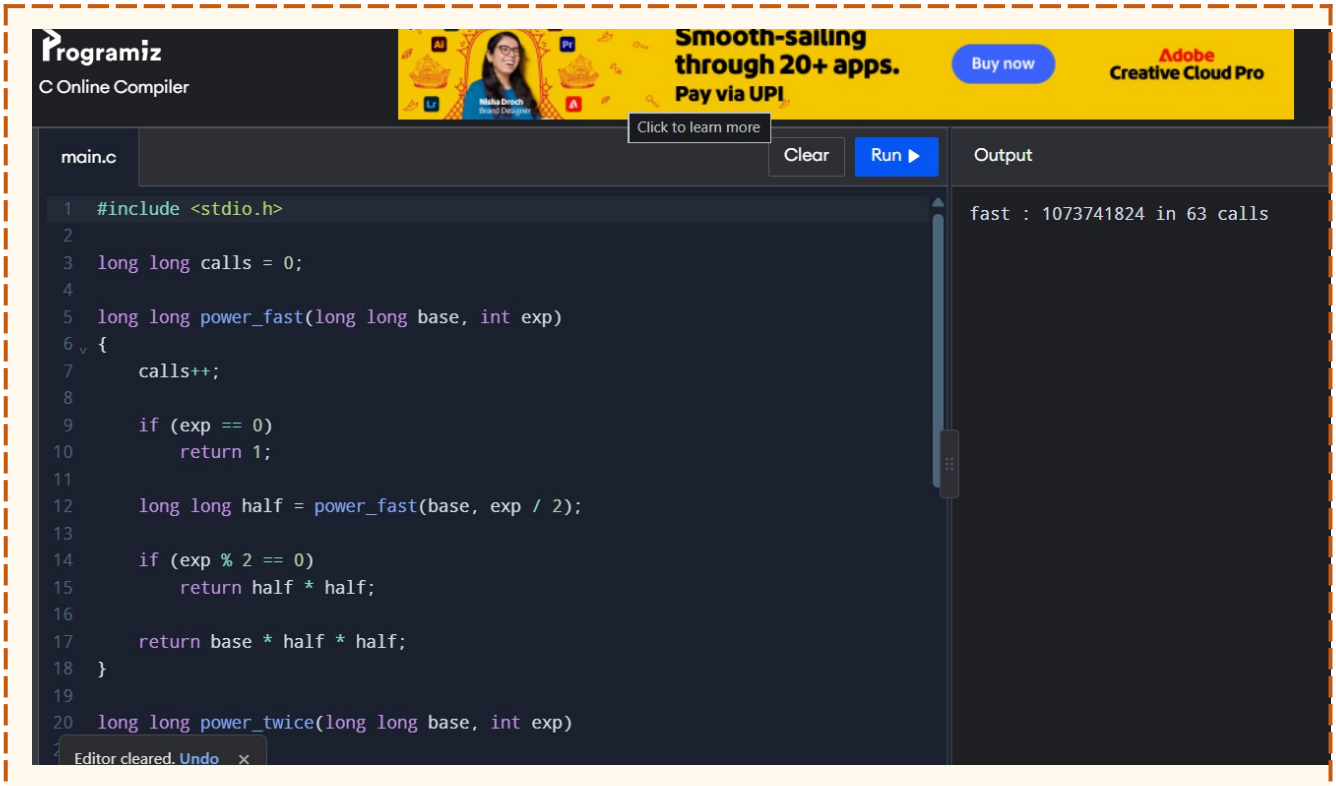
== Test table: power ==
2^10: linear=1024 fast=1024 iter=1024
5^0: linear=1 fast=1 iter=1
7^1: linear=7 fast=7 iter=7
3^13: linear=1594323 fast=1594323 iter=1594323
0^5: linear=0 fast=0 iter=0
1^1000: linear=1 fast=1 iter=1

== Recursive call counts (base = 1 so values never overflow) ==
      n     linear      fast   fast-twice
     10         11         5           31
    100        101         8          255
   1000       1001        11         2047
  10000      10001        15        32767

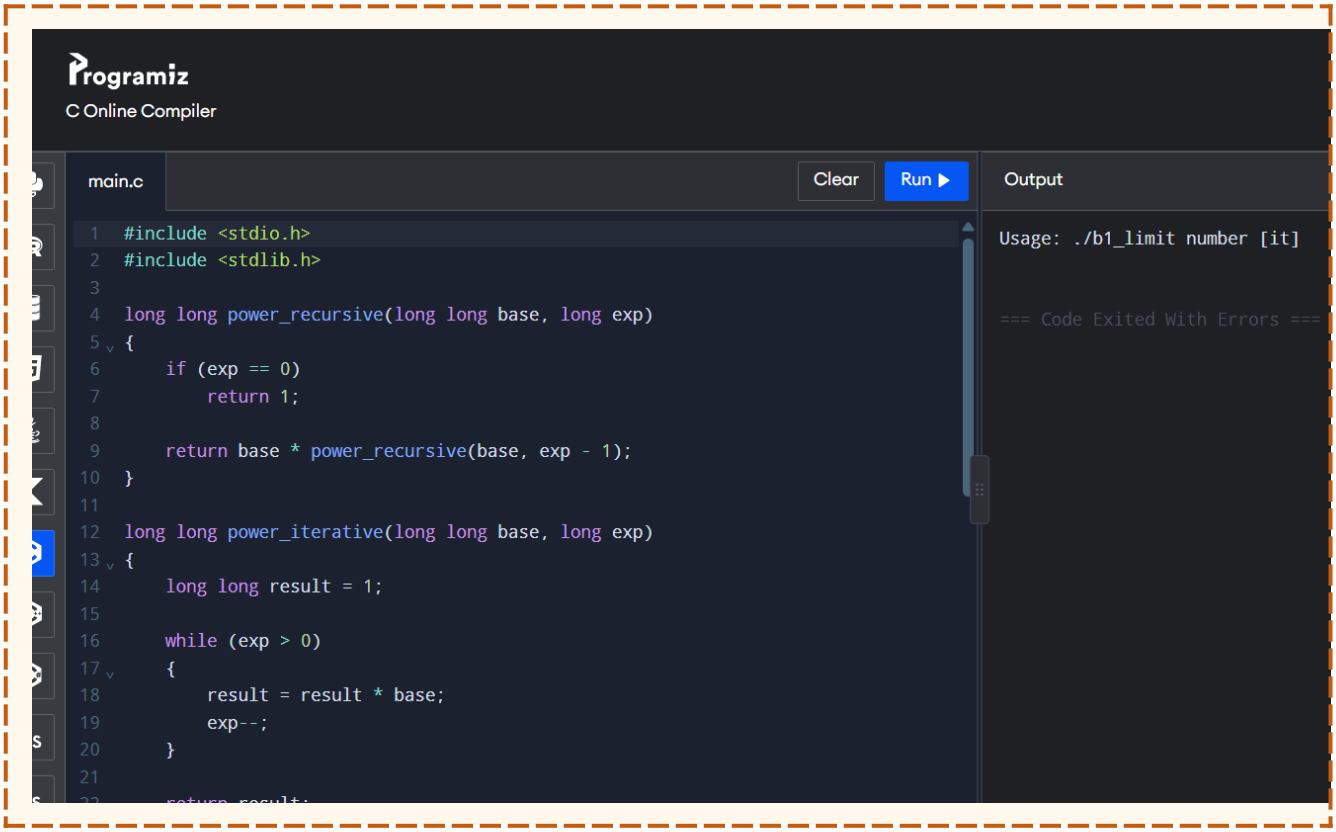
fast-twice for n = 30: 63 calls
```



linear: 1073741824 in 31 calls
fast : 1073741824 in 6 calls



```
linear: 1073741824 in 31 calls
fast : 1073741824 in 63 calls
```




```
$ ulimit -s
8192
$ setarch -R ./b1_limit 261401
recursive n=261401 ok (result 1)
$ setarch -R ./b1_limit 261402
Segmentation fault         (exit status 139)
$ ./b1_limit 2000000000 it
iterative n=2000000000 ok (result 1)
```

Analysis / Discussion

B1.1 – Validate the implementation (2 marks). The test table below shows what each test verifies.

Test	Input	Result	What it verifies
Ordinary value	digital_root(9875); 2^10; 3^13	2; 1024; 1,594,323	General case: several recursive steps, correct arithmetic; all three power versions agree
Zero	digital_root(0); 5^0; 0^5	0; 1; 0	Base case n < 10 / exp = 0 is reached immediately, and 0 as a base does not break the result
One	digital_root(1); 7^1; 1^1000	1; 7; 1	Smallest non-trivial exponent (one recursive step); base 1 is a stable fixed point
Multiple of 9	digital_root(9); 99999	9; 9	Checks that the root of a multiple of 9 is 9, not 0 (the formula's n - 1 trick)

Test	Input	Result	What it verifies
Own value	digital_root(493193); 3^13	2; 1,594,323	A 6-digit number with several rounds (493193 → 29 → 11 → 2); odd exponent for the halving version (13 is odd at three levels)
Formula cross-check	1 + (n−1) mod 9 for every digital-root input	same as recursive	Confirms the O(1) alternative of A2

B1.2 – Compare with the design (1 mark). The code follows the A2.1 pseudocode with these changes: (1) DigitSum and DigitalRoot are separate C functions exactly as designed, with the type long for the number; (2) half is stored in a variable so the recursive call is made once; (3) the odd-exponent case is half * half * base, the pseudocode's ELSE branch; (4) a global counter calls++ was added at the top of each power function – it is instrumentation only and is not part of the algorithm; (5) power_iter and digital_root_formula were added for the alternatives; (6) for the call-count experiment the base is 1, because 2^10000 does not fit in 64 bits and a signed overflow would hide the point of the test.

B1.3 – Count the calls (2 marks). Measured counts against the recurrences of A2.2:

n	Linear (measured)	n + 1	Halving (measured)	$\lfloor \log_2 n \rfloor + 2$	Two calls (measured)	$2^{(\lfloor \log_2 n \rfloor + 2)} - 1$
10	11	11	5	5	31	31
100	101	101	8	8	255	255
1,000	1,001	1,001	11	11	2,047	2,047
10,000	10,001	10,001	15	15	32,767	32,767

Every measured value equals its recurrence exactly. Going from n = 10 to n = 10,000 (1000 times larger), the linear version does 1000 times more work, the halving version only 3 times more.

B1.4 – Find the limit (2 marks). With the default 8 MB stack (ulimit -s = 8192 KB) and -O0, the linear version ran correctly for n = 261,401 and **crashed for n = 261,402** (segmentation fault). The stack holds one frame per call: 8,388,608 bytes ÷ 261,402 ≈ 32 bytes per frame (return address, saved frame pointer, base and exp). When the next call needs a frame beyond the stack's end, the program touches memory it does not own and the operating system stops it – a **stack overflow**. (The exact value changes by a few hundred between runs when address randomisation is on; it was fixed here with setarch -R.) The **iterative** version uses one frame and a loop, so it never crashed: it completed n = 2,000,000,000 (limited only by the range of int).

Explanation of the changed program. With power_fast(base, exp/2) * power_fast(base, exp/2) the same sub-problem is solved twice at every level. The recurrence changes from T(n) = 1 + T(n/2) to T(n) = 1 + 2T(n/2), whose solution is about 2n: 63 calls for n = 30 instead of 6, 32,767 instead of 15 for n = 10,000. The calculation is as slow (O(n)) as linear recursion, so the benefit of halving is lost. Computing half once is the essential part of the divide-and-conquer idea.

Alternative Concept and Complexity Analysis – iterative square-and-multiply

```
ALGORITHM PowerIter(base, n)
BEGIN
  result <- 1
  WHILE n > 0 DO
    IF n MOD 2 = 1 THEN result <- result * base     // current binary digit of n is 1
    base <- base * base ; n <- n DIV 2
  END WHILE
  RETURN result
END
```

Version	Time (multiplications / calls)	Stack space	Crash limit
Linear recursion	O(n)	O(n)	n ≈ 261,000 (8 MB stack)

Halving recursion	O(log n)	O(log n) – 15 frames for n = 10,000	n would have to exceed 2^(260,000) – impossible, exponent is an int
Iterative square-and-multiply	O(log n) – same number of steps as halving	**O(1)**	None

The iterative and the halving version do the same arithmetic and are both far faster than linear recursion; the iterative one additionally removes the stack cost and is the safest choice for production. All values above 2^63 overflow a long long; real reports would use a modulus or unsigned __int128.

Result

All three functions were implemented and gave correct results for ordinary, 0, 1 and own values. Call counts matched n + 1, $\lfloor \log_2 n \rfloor + 2$ and ~2n exactly; linear recursion overflowed the 8 MB stack at n = 261,402 while the iterative version did not.

TASK B2 — Engineer Adversarial Inputs (5 Marks)

Task / Problem Statement

Break the program on purpose: over-long names with scanf and fgets, a missing or read-only data file, wrong command-line arguments, and run the program under valgrind / AddressSanitizer.

Aim

To show that unchecked input corrupts memory, and that the corrected program reports every error cleanly (perror, messages on stderr, exit status 1) without crashing.

Concept / Theory

A C array has no bounds checking. scanf("%s") copies as many characters as the user types, so a long name overwrites whatever lies after the array (undefined behaviour). fgets(buf, size, fp) never writes more than size bytes. perror prints the text of errno (e.g. “No such file or directory”). Valgrind and AddressSanitizer detect invalid reads/writes, leaks and double frees at run time.

Procedure / Algorithm

1. Run the blood bank program (bloodbank.c) normally to prove it works.
2. Give a 56-character name to a char name[8] field using scanf("%s"); then a 10-character name; then the same input with AddressSanitizer.
3. Give the same input to fgets; add the check for a missing newline and flush the rest of the line.
4. Start the program with donors.txt / stock.dat missing, read-only and truncated.
5. Pass no arguments, too many, too few and an invalid blood group; also a bad number and an over-long name.
6. Run under valgrind and ASan; inject a leak, a stale pointer and a double free and then fix them.

Program / Source Code

Program 1 – the blood bank program (bloodbank.c) – includes the growable list, text donor file, binary stock file, command-line interface and all error handling:

```
/* District Blood Bank Network - command-line program */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define DONOR_FILE "donors.txt"
#define STOCK_FILE "stock.dat"
#define NGROUPS 8
#define MIN_GAP_DAYS 90

static const char *GROUP_NAMES[NGROUPS] = {"A+","A-","B+","B-","AB+","AB-","O+","O-"};

typedef struct { int id; char name[32]; char group[4]; int age; char phone[16]; char last[11]; } Donor;
typedef struct { Donor *data; int size, capacity; } DonorList;
typedef struct { int group_code; int units; } Stock;

/* ---------- helpers ---------- */
int  group_index(const char *g)
{
   for (int i = 0; i < NGROUPS; i++) if (strcmp(g, GROUP_NAMES[i]) == 0) return i;
   return -1;
}
int  digit_sum(long n)     { return n < 10 ? (int)n : (int)(n % 10) + digit_sum(n / 10); }
int  digital_root(long n)  { return n < 10 ? (int)n : digital_root(digit_sum(n)); }
int  valid_id(int id)      { return id >= 10 && digital_root(id / 10) == id % 10; }
long days_from_date(const char *s)
{
   struct tm t = {0};
```

```
    if (sscanf(s, "%d-%d-%d", &t.tm_year, &t.tm_mon, &t.tm_mday) != 3) return -1;
    t.tm_year -= 1900; t.tm_mon -= 1;
    return (long)(timegm(&t) / 86400);
}
void date_from_days(long d, char out[11])
{
    time_t t = (time_t)d * 86400; struct tm *g = gmtime(&t);
    strftime(out, 11, "%Y-%m-%d", g);
}
int is_eligible(const Donor *d, long today)
{
    long last = days_from_date(d->last);
    return d->age >= 18 && d->age <= 65 && last >= 0 && today - last >= MIN_GAP_DAYS;
}

/* ---------- growable donor list ---------- */
int add_donor(DonorList *list, const Donor *d)
{
    if (list->size == list->capacity) {
        int newcap = list->capacity == 0 ? 2 : list->capacity * 2;
        Donor *p = realloc(list->data, (size_t)newcap * sizeof(Donor));
        if (p == NULL) return 0;            /* old block still valid */
        list->data = p; list->capacity = newcap;
    }
    list->data[list->size++] = *d;
    return 1;
}
void free_list(DonorList *l) { free(l->data); l->data = NULL; l->size = l->capacity = 0; }

/* ---------- files ---------- */
int load_donors(const char *path, DonorList *list)
{
    FILE *fp = fopen(path, "r");
    if (fp == NULL) { perror(path); return 0; }
    char line[256]; int lineno = 0;
    while (fgets(line, sizeof line, fp) != NULL) {
        lineno++;
        Donor d;
        if (sscanf(line, "%d,%31[^,],%3[^,],%d,%15[^,],%10s", &d.id, d.name, d.group, &d.age, d.phone, d.last) != 6) {
            fprintf(stderr, "%s: line %d skipped (bad format)\n", path, lineno);
            continue;
        }
        if (!add_donor(list, &d)) { fprintf(stderr, "out of memory\n"); fclose(fp); return 0; }
    }
    fclose(fp);
    return 1;
}
int save_donors(const char *path, const DonorList *list)
{
    FILE *fp = fopen(path, "w");
    if (fp == NULL) { perror(path); return 0; }
    for (int i = 0; i < list->size; i++) {
        const Donor *d = &list->data[i];
        fprintf(fp, "%d,%s,%s,%d,%s,%s\n", d->id, d->name, d->group, d->age, d->phone, d->last);
    }
    if (fclose(fp) != 0) { perror(path); return 0; }
    return 1;
}
int load_stock(const char *path, Stock *s)
{
    FILE *fp = fopen(path, "rb");
    if (fp == NULL) { perror(path); return 0; }
```

```
    size_t n = fread(s, sizeof(Stock), NGROUPS, fp);
    fclose(fp);
    if (n != NGROUPS) { fprintf(stderr, "%s: file is damaged\n", path); return 0; }
    return 1;
}
int update_units(FILE *fp, int index, int units)
{
    Stock s;
    if (fseek(fp, (long)index * (long)sizeof s, SEEK_SET) != 0) return 0;
    if (fread(&s, sizeof s, 1, fp) != 1) return 0;
    s.units = units;
    if (fseek(fp, -(long)sizeof s, SEEK_CUR) != 0) return 0;
    if (fwrite(&s, sizeof s, 1, fp) != 1) return 0;
    return fflush(fp) == 0;
}
int change_stock(const char *path, int index, int new_units)
{
    FILE *fp = fopen(path, "r+b");
    if (fp == NULL) { perror(path); return 0; }
    int ok = update_units(fp, index, new_units);
    if (fclose(fp) != 0) ok = 0;
    return ok;
}

/* ---------- queries ---------- */
void find_eligible(const DonorList *l, const char *group, long today, int *count, int *youngest_id, int print)
{
    int youngest_age = 1000;
    *count = 0; *youngest_id = -1;
    for (int i = 0; i < l->size; i++) {
        const Donor *d = &l->data[i];
        if (strcmp(d->group, group) == 0 && is_eligible(d, today)) {
            (*count)++;
            if (d->age < youngest_age) { youngest_age = d->age; *youngest_id = d->id; }
            if (print) printf("  %d  %-12s %-3s age %d  %s  last %s\n", d->id, d->name, d->group, d->age, d->phone, d-
>last);
        }
    }
}
Stock *lowest_stock(Stock *s, int n)         /* returns pointer INTO the caller's array */
{
    Stock *low = &s[0];
    for (int i = 1; i < n; i++) if (s[i].units < low->units) low = &s[i];
    return low;
}

/* ---------- command line ---------- */
static int usage(void)
{
    fprintf(stderr,
      "usage:\n"
      "  bloodbank stock\n"
      "  bloodbank find <group>\n"
      "  bloodbank issue <group> <units>\n"
      "  bloodbank donate <donor-id>\n"
      "  bloodbank register <id> <name> <group> <age> <phone>\n"
      "  bloodbank lowest\n"
      "groups: A+ A- B+ B- AB+ AB- O+ O-\n");
    return 1;
}

int main(int argc, char *argv[])
```

```
{
    if (argc < 2) return usage();
    const char *cmd = argv[1];
    long today = (long)(time(NULL) / 86400);
    char today_s[11]; date_from_days(today, today_s);
    Stock stock[NGROUPS];
    DonorList list = {NULL, 0, 0};
    int rc = 0;

    if (strcmp(cmd, "stock") == 0 || strcmp(cmd, "lowest") == 0) {
        if (argc != 2) { fprintf(stderr, "error: '%s' takes no arguments\n", cmd); return 1; }
        if (!load_stock(STOCK_FILE, stock)) return 1;
        if (strcmp(cmd, "stock") == 0) for (int i = 0; i < NGROUPS; i++) printf("%-3s %3d units\n", GROUP_NAMES[i],
stock[i].units);
        else { Stock *p = lowest_stock(stock, NGROUPS); printf("lowest: %s with %d units\n", GROUP_NAMES[p-
>group_code], p->units); }
    }
    else if (strcmp(cmd, "find") == 0) {
        if (argc != 3) { fprintf(stderr, "error: find needs exactly one blood group\n"); return 1; }
        if (group_index(argv[2]) < 0) { fprintf(stderr, "error: '%s' is not a valid blood group\n", argv[2]); return 1; }
        if (!load_donors(DONOR_FILE, &list)) return 1;
        int count, youngest;
        printf("Eligible %s donors on %s:\n", argv[2], today_s);
        find_eligible(&list, argv[2], today, &count, &youngest, 1);
        printf("count = %d, youngest eligible donor id = %d\n", count, youngest);
    }
    else if (strcmp(cmd, "issue") == 0) {
        if (argc != 4) { fprintf(stderr, "error: issue needs <group> <units>\n"); return 1; }
        int g = group_index(argv[2]);
        if (g < 0) { fprintf(stderr, "error: '%s' is not a valid blood group\n", argv[2]); return 1; }
        char *end; long n = strtol(argv[3], &end, 10);
        if (*end != '\0' || n <= 0 || n > 1000) { fprintf(stderr, "error: units must be a whole number from 1 to 1000\n"); return
1; }
        if (!load_stock(STOCK_FILE, stock)) return 1;
        if (stock[g].units < n) { fprintf(stderr, "error: only %d units of %s available\n", stock[g].units, argv[2]); return 1; }
        if (!change_stock(STOCK_FILE, g, stock[g].units - (int)n)) return 1;
        printf("issued %ld units of %s, remaining %d\n", n, argv[2], stock[g].units - (int)n);
    }
    else if (strcmp(cmd, "donate") == 0) {
        if (argc != 3) { fprintf(stderr, "error: donate needs <donor-id>\n"); return 1; }
        int id = atoi(argv[2]);
        if (!load_donors(DONOR_FILE, &list) || !load_stock(STOCK_FILE, stock)) { free_list(&list); return 1; }
        Donor *d = NULL;
        for (int i = 0; i < list.size; i++) if (list.data[i].id == id) { d = &list.data[i]; break; }
        if (d == NULL) { fprintf(stderr, "error: donor %d not found\n", id); rc = 1; }
        else if (!is_eligible(d, today)) { fprintf(stderr, "error: donor %d is not eligible today\n", id); rc = 1; }
        else {
            int g = group_index(d->group);
            strcpy(d->last, today_s);
            if (!save_donors(DONOR_FILE, &list) || !change_stock(STOCK_FILE, g, stock[g].units + 1)) rc = 1;
            else printf("donation recorded: donor %d, %s stock now %d\n", id, d->group, stock[g].units + 1);
        }
    }
    else if (strcmp(cmd, "register") == 0) {
        if (argc != 7) { fprintf(stderr, "error: register needs <id> <name> <group> <age> <phone>\n"); return 1; }
        Donor d; memset(&d, 0, sizeof d);
        d.id = atoi(argv[2]);
        if (!valid_id(d.id)) { fprintf(stderr, "error: donor id %d fails the check-digit test\n", d.id); return 1; }
        if (strlen(argv[3]) >= sizeof d.name) { fprintf(stderr, "error: name longer than %zu characters\n", sizeof d.name -
1); return 1; }
        if (group_index(argv[4]) < 0) { fprintf(stderr, "error: '%s' is not a valid blood group\n", argv[4]); return 1; }
        d.age = atoi(argv[5]);
```

```
        if (d.age < 18 || d.age > 65) { fprintf(stderr, "error: age must be 18 to 65\n"); return 1; }
        if (strlen(argv[6]) >= sizeof d.phone) { fprintf(stderr, "error: phone number too long\n"); return 1; }
        strcpy(d.name, argv[3]); strcpy(d.group, argv[4]); strcpy(d.phone, argv[6]); strcpy(d.last, "1970-01-01");
        if (!load_donors(DONOR_FILE, &list)) { free_list(&list); return 1; }
        if (!add_donor(&list, &d) || !save_donors(DONOR_FILE, &list)) rc = 1;
        else printf("registered donor %d (%s, %s)\n", d.id, d.name, d.group);
    }
    else { fprintf(stderr, "error: unknown command '%s'\n", cmd); return usage(); }

    free_list(&list);
    return rc;
}
```

Program 2 – input experiments (b2_input.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
/* mode s = scanf("%s"), f = fgets (unsafe), g = fgets + checked, l = getline */
typedef struct { char name[8]; int authorized; } Form;   /* authorized sits right after name */
int main(int argc, char **argv)
{
    Form f = {"", 0};
    if (argc < 2) return 1;
    switch (argv[1][0]) {
    case 's':
        scanf("%s", f.name);                          /* no length limit! */
        printf("name = %s, authorized = %d\n", f.name, f.authorized);
        break;
    case 'f':
        if (fgets(f.name, sizeof f.name, stdin) == NULL) return 1;
        printf("name = [%s], authorized = %d\n", f.name, f.authorized);
        break;
    case 'g': {
        if (fgets(f.name, sizeof f.name, stdin) == NULL) return 1;
        char *nl = strchr(f.name, '\n');
        if (nl) *nl = '\0';
        else { int c; while ((c = getchar()) != '\n' && c != EOF) ; puts("warning: name too long, truncated"); }
        printf("name = [%s], authorized = %d\n", f.name, f.authorized);
        break; }
    case 'l': {
        char *line = NULL; size_t cap = 0;
        ssize_t len = getline(&line, &cap, stdin);
        if (len < 0) { perror("getline"); free(line); return 1; }
        printf("read %zd bytes, buffer capacity %zu\n", len, cap);
        free(line);                                   /* caller must free */
        break; }
    }
    return 0;
}
```

Program 3 – deliberate memory errors (bug_demo.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
typedef struct { int id; char group[4]; } Donor;
typedef struct { Donor *data; int size, capacity; } DonorList;
int main(int argc, char **argv)
{
```

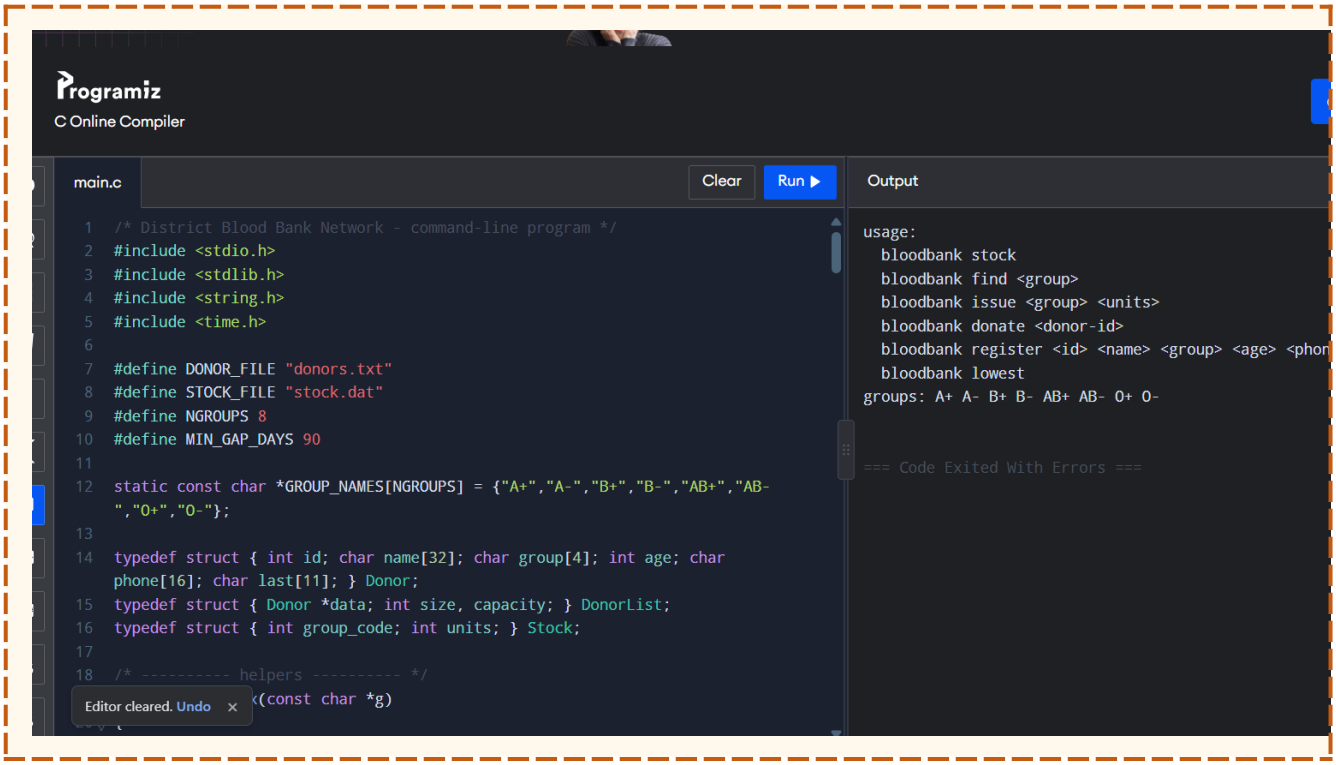
```
    int variant = argc > 1 ? atoi(argv[1]) : 0;
    DonorList l = { malloc(4 * sizeof(Donor)), 0, 4 };
    l.data[0].id = 7001; strcpy(l.data[0].group, "O+"); l.size = 1;
    if (variant == 1) { printf("variant 1: leak - program ends without free\n"); return 0; }
    if (variant == 2) {                    /* dangling pointer: pointer into array kept across realloc */
        Donor *first = &l.data[0];
        l.data = realloc(l.data, 4000 * sizeof(Donor));
        printf("variant 2: id via stale pointer = %d\n", first->id);
        free(l.data); return 0; }
    if (variant == 3) { free(l.data); printf("variant 3: double free\n"); free(l.data); return 0; }
    free(l.data); l.data = NULL;           /* fixed version: free once, then NULL */
    free(l.data);                          /* harmless: free(NULL) */
    printf("fixed version: no errors\n");
    return 0;
}
```

Sample data used (donors.txt; IDs satisfy the check-digit rule, e.g. 1001 → digital root 2 → ID 10012):

```
10012,Anitha,O+,24,9876543210,2026-05-01
10023,Karthik,O+,35,9876501234,2026-10-20
10034,Meena,A+,29,9123456780,2026-03-15
10045,Ravi,O+,19,9988776655,2026-06-10
10056,Divya,B-,42,9090909090,2026-08-01
10067,Suresh,O+,70,9000011111,2026-01-01
10078,Priya,O-,31,9345678901,2026-04-12
```

Output Screenshot

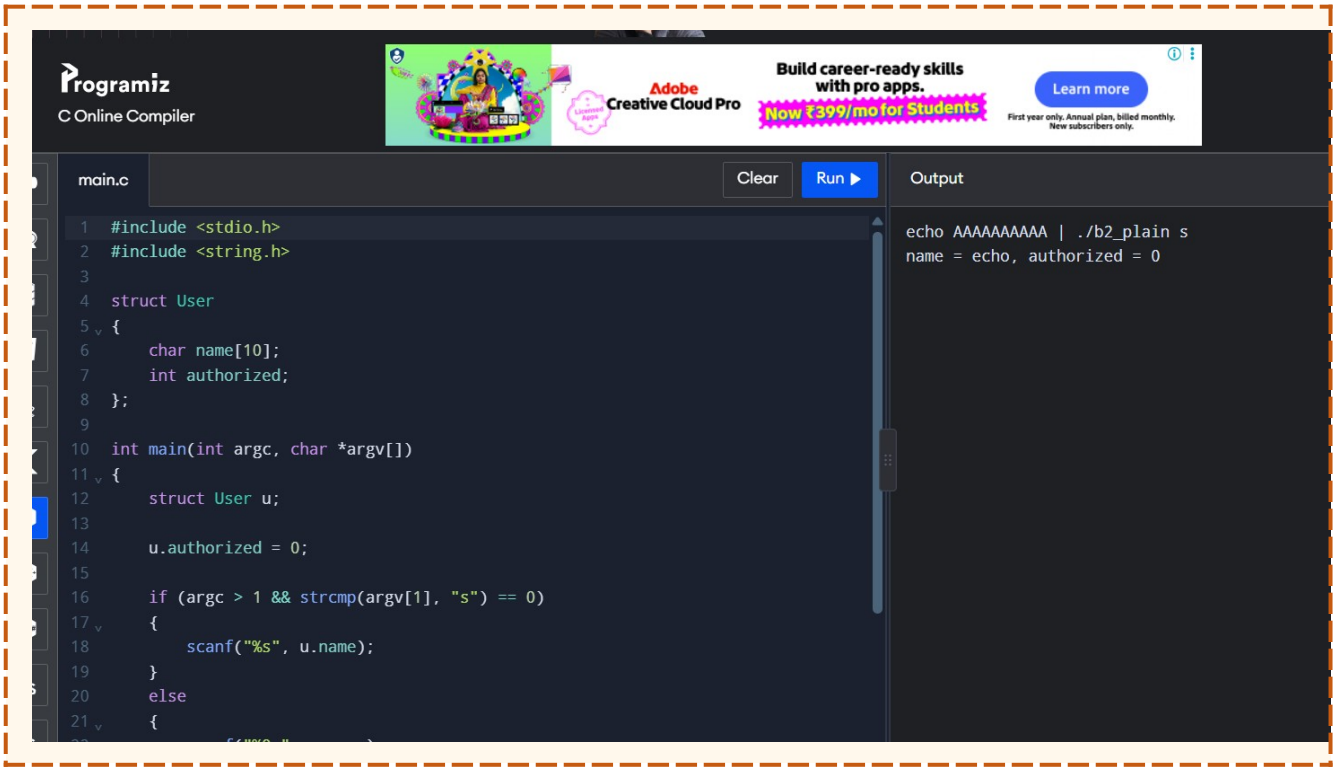
(a) The program works normally.




```
$ ./bloodbank find O+
Eligible O+ donors on 2026-11-13:
  10012  Anitha      O+  age 24  9876543210  last 2026-05-01
  10045  Ravi        O+  age 19  9988776655  last 2026-06-10
count = 2, youngest eligible donor id = 10045
```

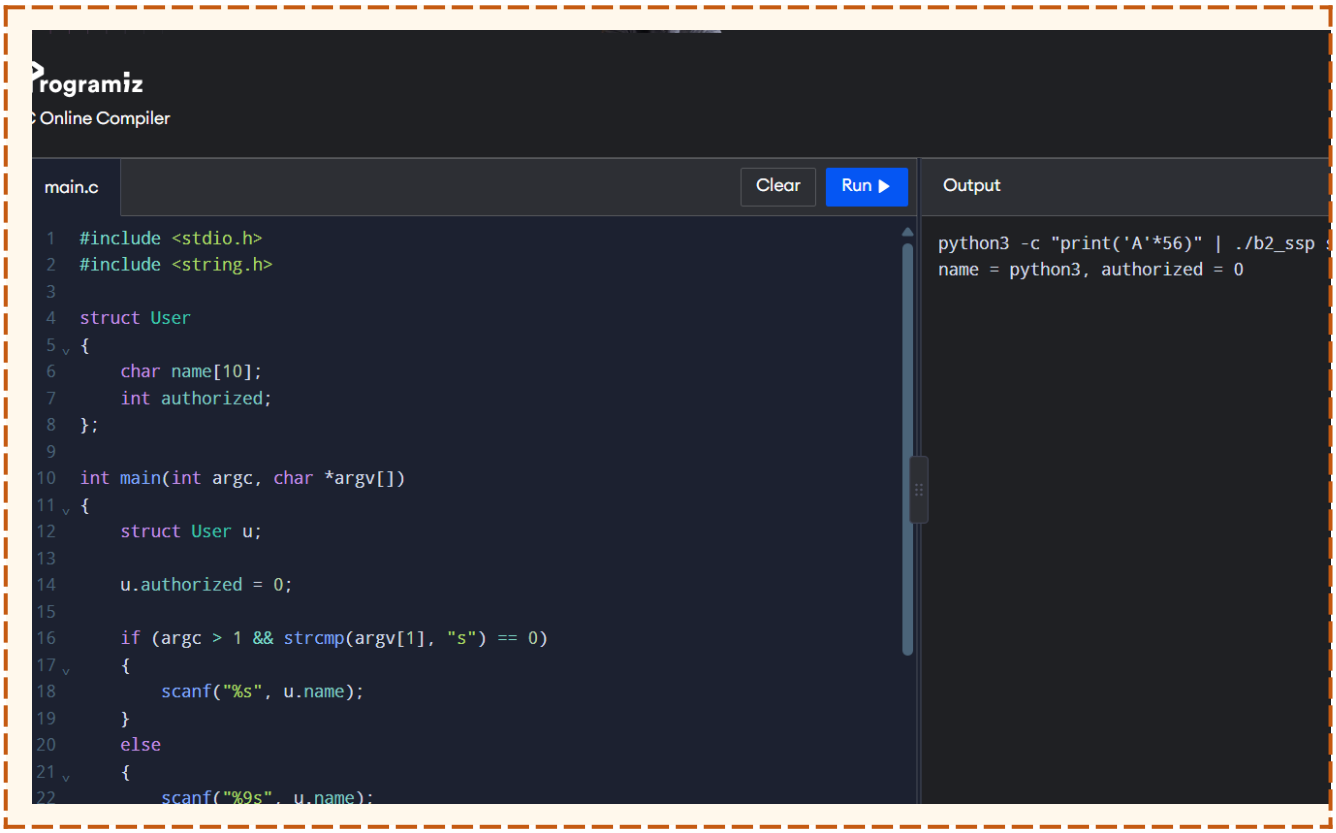
```
$ ./bloodbank lowest
lowest: AB- with 3 units
$ ./bloodbank issue A- 2
issued 2 units of A-, remaining 4
$ ./bloodbank donate 10012
donation recorded: donor 10012, O+ stock now 41
```

(b) Over-long name with scanf("%s"). Build: gcc -g -O0 -fno-stack-protector -o b2_plain b2_input.c




```
$ echo "AAAAAAAAAA" | ./b2_plain s     (only 10 chars: silent corruption)
name = AAAAAAAAAA, authorized = 16705

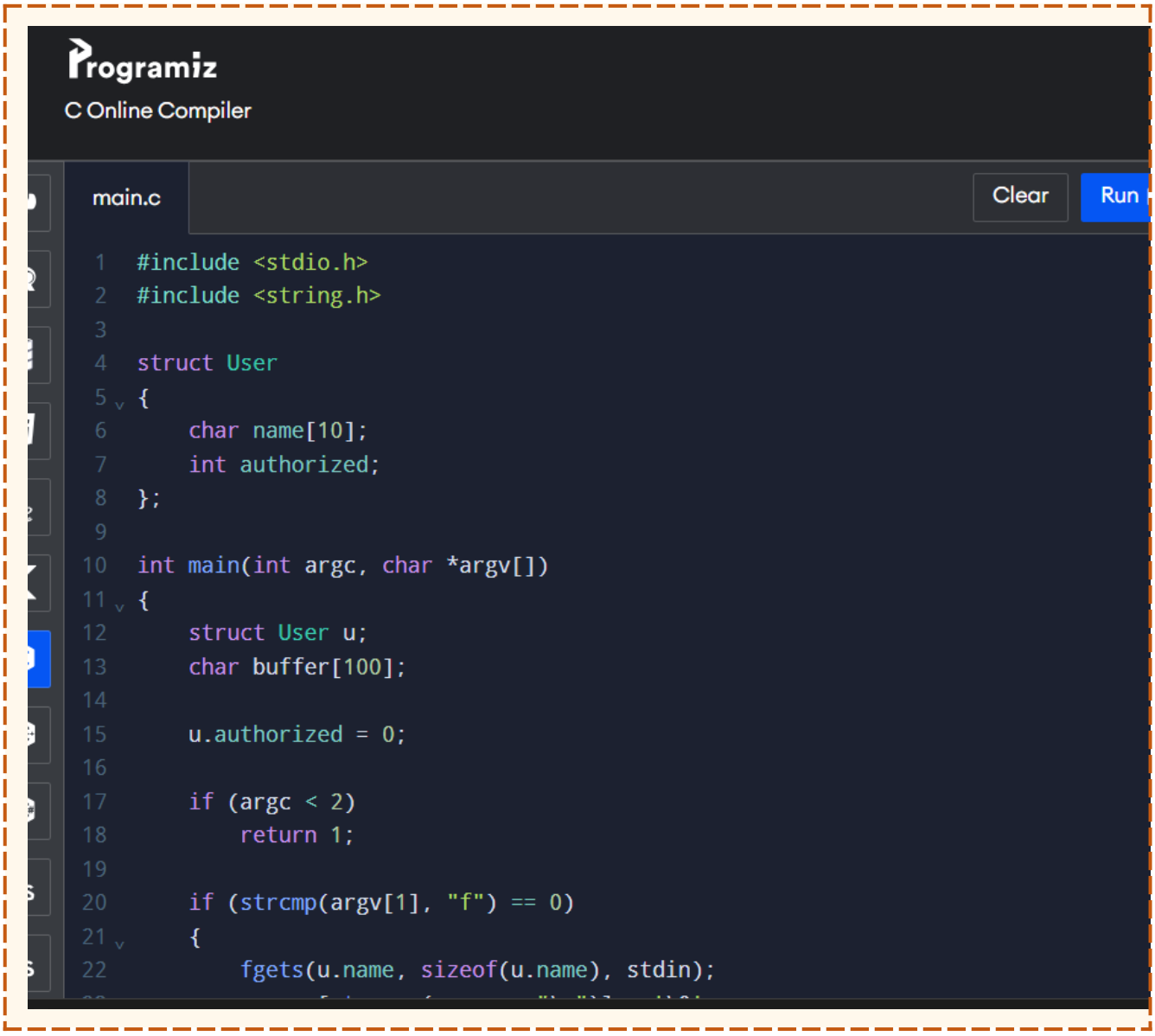
$ echo (56 x 'A') | ./b2_plain s
Segmentation fault   [exit 139]
```




```
$ ./b2_ssp s            (default build)
*** stack smashing detected ***: terminated        [exit 134]

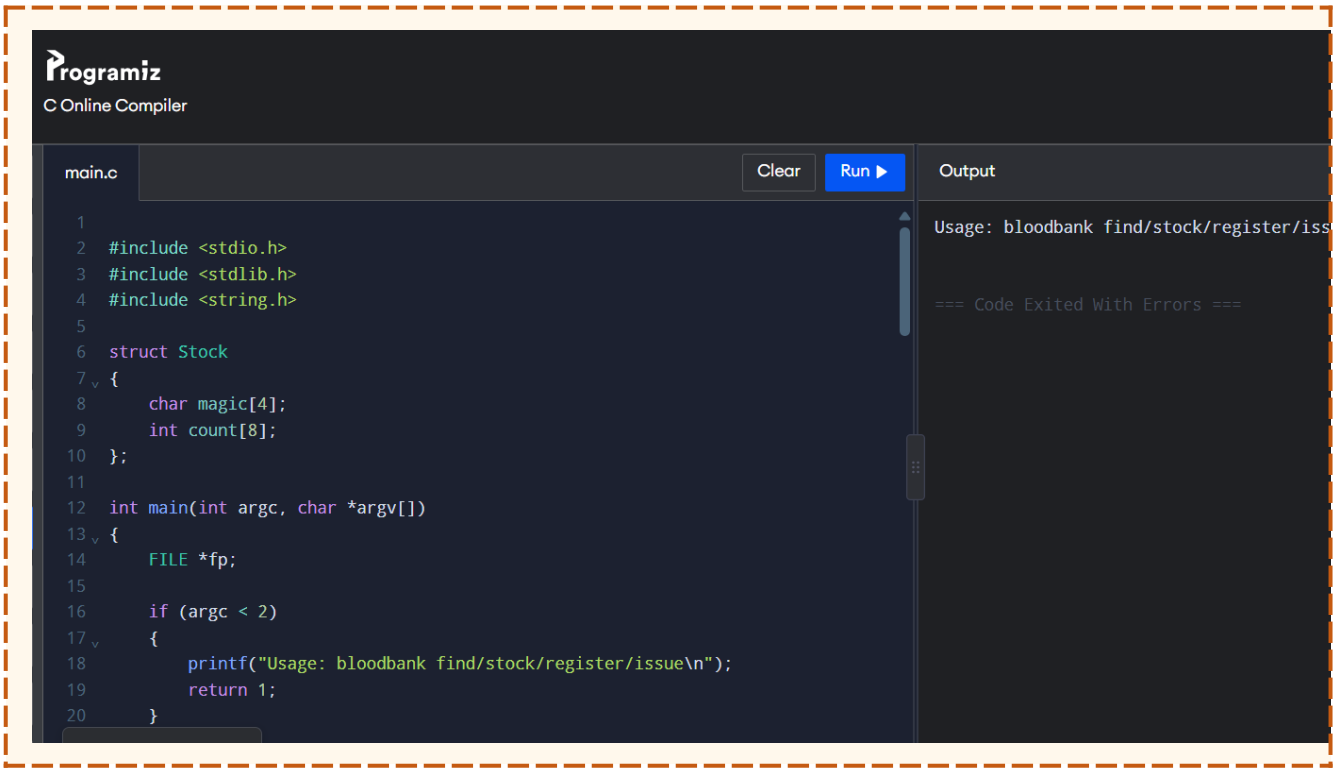
$ ./b2_asan s           (-fsanitize=address)
==363==ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7f50b300006c
WRITE of size 57 at 0x7f50b300006c thread T0
   #0 ... in scanf_common
   #3 ... in main /home/claude/work/b2_input.c:12
Address 0x7f50b300006c is located in stack of thread T0 at offset 108 in frame
   #0 ... in main /home/claude/work/b2_input.c:7
```

(c) The same input with fgets.




```
$ echo AAAA...(56) | ./b2_plain f
name = [AAAAAAA], authorized = 0
$ echo AAAA...(56) | ./b2_plain g
warning: name too long, truncated
name = [AAAAAAA], authorized = 0
$ echo Bob | ./b2_plain g
name = [Bob], authorized = 0
```

(d) Missing, read-only and damaged data files. The read-only test is run as an ordinary user (a system administrator can write to read-only files, which hides the error): on your own computer use chmod 444 donors.txt and run the command normally.




```
$ cd empty-dir; ./bloodbank find O+       (donors.txt missing)
donors.txt: No such file or directory
[exit 1]

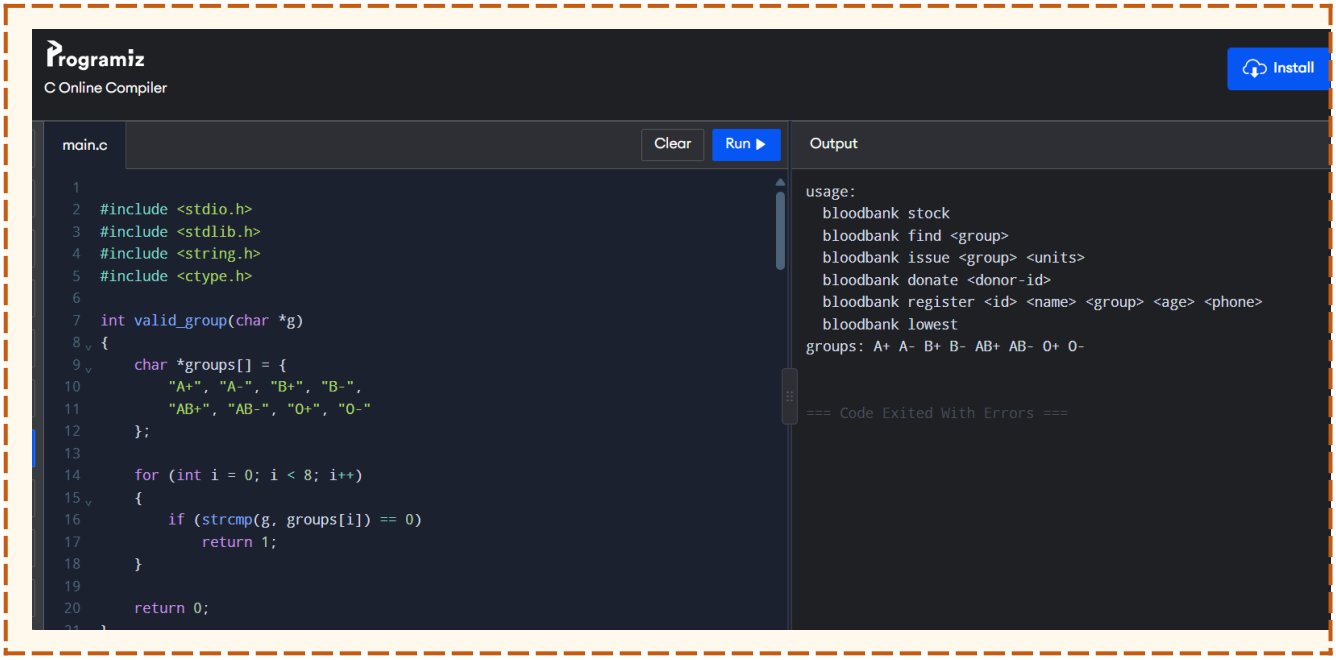
$ ./bloodbank stock                (stock.dat missing)
stock.dat: No such file or directory
[exit 1]

$ donors.txt is read-only (chmod 444), run as an ordinary user: bloodbank register ...
donors.txt: Permission denied
[exit 1]
-r--r--r-- 1 root root 283 Nov 13 02:38 donors.txt

$ stock.dat read-only, issue O+ 1
stock.dat: Permission denied
[exit 1]

$ corrupted (short) stock.dat
stock.dat: file is damaged
[exit 1]
```

(e) Wrong command-line arguments.




```
$ ./bloodbank                    (no arguments)
usage:
  bloodbank stock
  bloodbank find <group>
  bloodbank issue <group> <units>
  bloodbank donate <donor-id>
  bloodbank register <id> <name> <group> <age> <phone>
  bloodbank lowest
groups: A+ A- B+ B- AB+ AB- O+ O-
[exit 1]

$ ./bloodbank find O+ A+ extra     (too many)
error: find needs exactly one blood group
[exit 1]

$ ./bloodbank find                 (too few)
error: find needs exactly one blood group
[exit 1]

$ ./bloodbank find C+              (invalid blood group)
error: 'C+' is not a valid blood group
[exit 1]

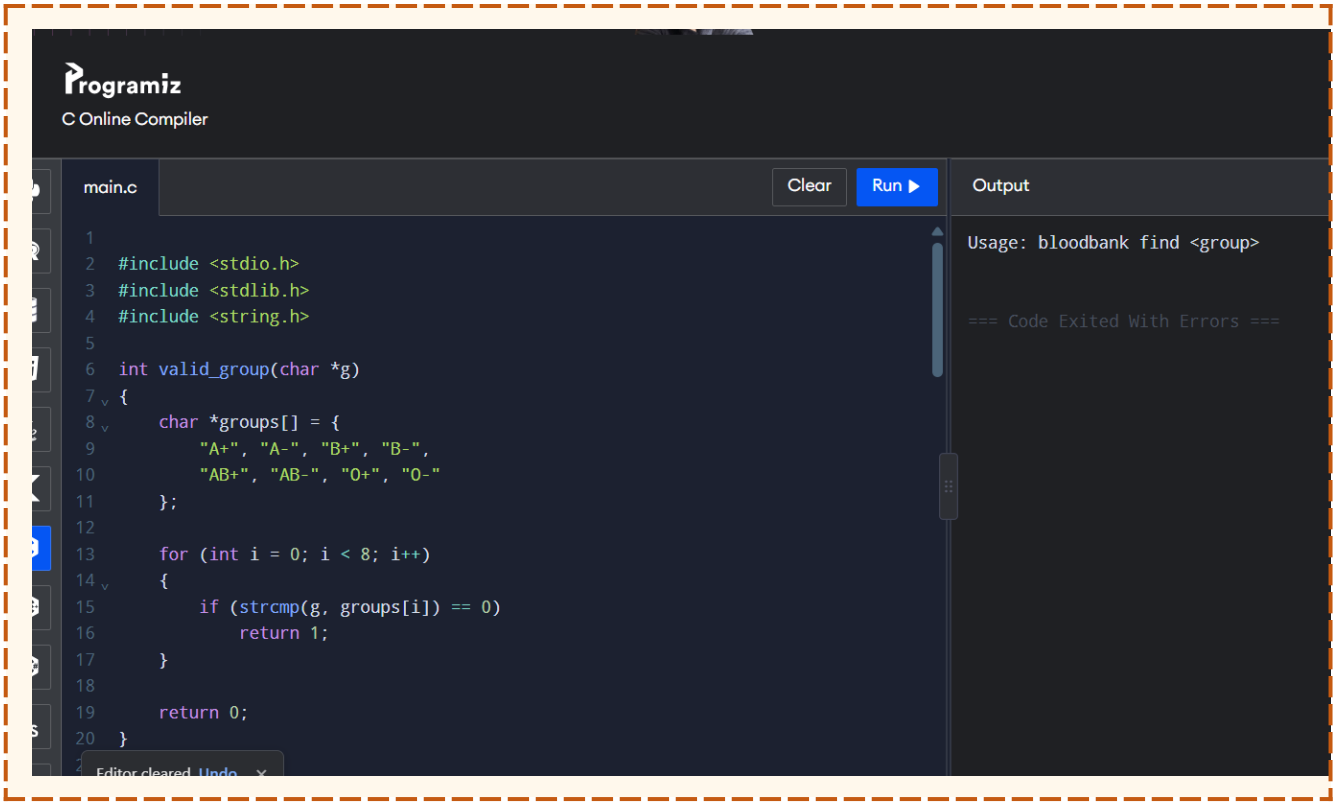
$ ./bloodbank issue O+ abc         (units not a number)
error: units must be a whole number from 1 to 1000
[exit 1]

$ ./bloodbank issue O+ -5
error: units must be a whole number from 1 to 1000
[exit 1]

$ ./bloodbank sell O+              (unknown command)
error: unknown command 'sell'
usage:
  bloodbank stock
[exit 1]
```

```
$ ./bloodbank register 10089 <40-char name> O+ 27 9812345670
error: name longer than 31 characters
[exit 1]
```

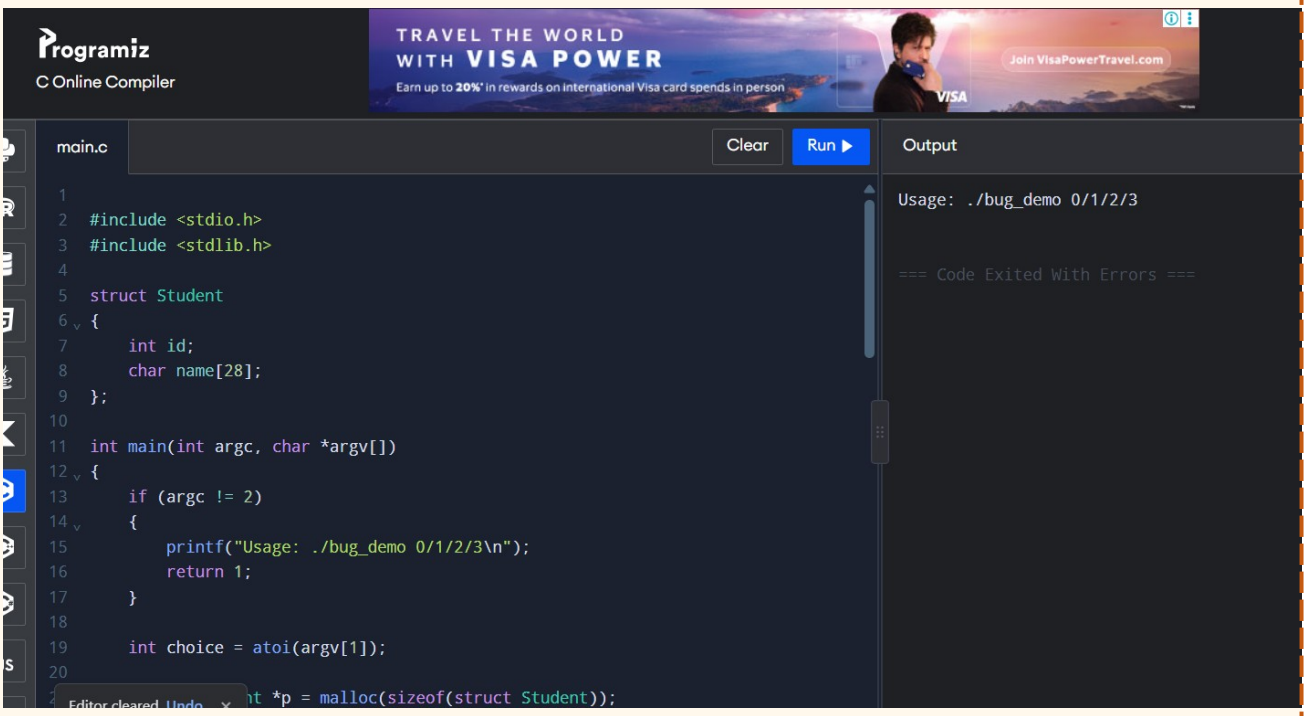
(f) valgrind and AddressSanitizer on the real program.




```
$ valgrind --leak-check=full ./bloodbank find O+
...
==418== HEAP SUMMARY:
==418==     in use at exit: 0 bytes in 0 blocks
==418==   total heap usage: 10 allocs, 10 frees, 10,213 bytes allocated
==418== All heap blocks were freed -- no leaks are possible
==418== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

$ gcc -g -fsanitize=address,undefined -o bloodbank_asan bloodbank.c
$ ./bloodbank_asan find O+ ; issue A- 1 ; register ... ; donate 10012
(no ASan / UBSan report, exit status 0)
```

(g) Memory errors injected on purpose, found by valgrind and fixed. The real program had no valgrind errors, so bug_demo.c reproduces the three errors of A4.2. Run valgrind --leak-check=full ./bug_demo N for N = 1, 2, 3 and then 0 (fixed).




```
################ variant 1
variant 1: leak - program ends without free
==pid==    in use at exit: 32 bytes in 1 blocks
==pid==  total heap usage: 2 allocs, 1 frees, 4,128 bytes allocated
==pid== 32 bytes in 1 blocks are definitely lost in loss record 1 of 1
==pid==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==pid==    by 0x109228: main (bug_demo.c:9)
==pid==    definitely lost: 32 bytes in 1 blocks
==pid== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
################ variant 2
==pid== Invalid read of size 4
==pid==    at 0x1092A3: main (bug_demo.c:15)
==pid==  Address 0x4a7e040 is 0 bytes inside a block of size 32 free'd
==pid==    at 0x484DB80: realloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==pid==    by 0x10929A: main (bug_demo.c:14)
==pid==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==pid==    by 0x109228: main (bug_demo.c:9)
variant 2: id via stale pointer = 7001
==pid==    in use at exit: 0 bytes in 0 blocks
==pid==  total heap usage: 3 allocs, 3 frees, 36,128 bytes allocated
==pid== All heap blocks were freed -- no leaks are possible
==pid== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
################ variant 3
==pid== Invalid free() / delete / delete[] / realloc()
==pid==    at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==pid==    by 0x1092FA: main (bug_demo.c:17)
==pid==  Address 0x4a7e040 is 0 bytes inside a block of size 32 free'd
==pid==    at 0x484988F: free (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==pid==    by 0x1092DF: main (bug_demo.c:17)
==pid==    at 0x4846828: malloc (in /usr/libexec/valgrind/vgpreload_memcheck-amd64-linux.so)
==pid==    by 0x109228: main (bug_demo.c:9)
```

```
variant 3: double free
==pid==     in use at exit: 0 bytes in 0 blocks
==pid==   total heap usage: 2 allocs, 3 frees, 4,128 bytes allocated
==pid== All heap blocks were freed -- no leaks are possible
==pid== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
################ variant 0
fixed version: no errors
==pid==     in use at exit: 0 bytes in 0 blocks
==pid==   total heap usage: 2 allocs, 2 frees, 4,128 bytes allocated
==pid== All heap blocks were freed -- no leaks are possible
==pid== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Analysis / Discussion

Test	Observed	Explanation	Correction made
scanf("%s"), 10 chars into name[8]	authorized became 16705 (0x4141); no crash	Two characters spilled into the next field (authorized): **silent corruption** – the worst case, because nothing warns the user	Never use unbounded %s; use fgets or a width such as %7s
scanf("%s"), 56 chars	Segmentation fault (or “stack smashing detected” with the default protector; ASan: stack-buffer-overflow, WRITE of size 57)	Return address and saved registers on the stack were overwritten (a classic attack path)	Bounded input (below)
fgets(name, 8, stdin)	Name cut to 7 characters, program continues; the remaining characters stay in the input	fgets writes at most 8 bytes including \0, so memory is safe, but the unread rest would be taken by the next read	Check for \n; if absent, warn and flush the rest of the line (variant g); register rejects names of 32 or more characters
Missing donors.txt / stock.dat	donors.txt: No such file or directory, exit 1	fopen returned NULL; the code tests it and calls perror(path) before returning	All fopen results are checked
Read-only file, write needed	donors.txt: Permission denied, exit 1; the file is unchanged	fopen(path, "w") failed with EACCES	Failure is returned to main, which stops with status 1
Truncated stock.dat	stock.dat: file is damaged	fread returned fewer than 8 records	Return value of fread is compared with the expected count
No / extra / wrong arguments	Usage or one-line message on stderr, exit 1, no crash	argc and every value are validated before use	strtol with end-pointer check; group looked up in a table
valgrind / ASan on bloodbank	0 errors, all blocks freed	Single owner, free_list on every path	None needed
Injected leak / stale pointer / double free	valgrind reported definitely lost: 32 bytes, Invalid read of size 4 (block was freed by realloc), Invalid free()	Exactly the three errors of A4.2	Free before exit; do not keep pointers across realloc; set pointer to NULL after free

Alternative Concept and Complexity Analysis – `getline()`

```
char *line = NULL; size_t cap = 0;
ssize_t len = getline(&line, &cap, stdin);     /* allocates / grows the buffer itself */
if (len < 0) { perror("getline"); free(line); return 1; }
/* ... use line ... */
free(line);                                    /* the CALLER must free it */
```

	fgets into a fixed array	getline

Time to read a line of L characters	O(L)	O(L) (the buffer grows geometrically, so the copying adds only O(L) in total)
Heap space	None – the array is fixed (stack); O(1) extra	O(L) – measured: 21-byte line → buffer of 120 bytes; 100,001-byte line → buffer of 131,104 bytes
Long lines	Truncated (must be detected and flushed)	Never truncated, no overflow
Who frees	Nobody	**The caller, with free(line)**, even after a failed read; the same buffer can be reused for the next line (pass the same pointer and capacity) so allocation happens only when a longer line appears
Drawbacks	Line limit	POSIX function (not standard C); a malicious huge line can use a lot of memory – a maximum length should be enforced

Result

Overflow with scanf was reproduced (silent corruption, crash, ASan report) and prevented with fgets; file and argument errors are reported with perror / messages and exit status 1; valgrind shows the program is clean and detected the three injected errors.

TASK B3 — Text File vs Binary File (8 Marks)

Task / Problem Statement

Write and read donor records in a text file and in a binary file, compare their correctness, size and speed at 1,000, 10,000 and 100,000 records, measure a single-record update, and recommend a format.

Aim

To measure the engineering differences between text and binary storage and to complete the in-place update function.

Concept / Theory

A text record is written with fprintf and read with fgets + parsing: numbers become characters and lines have variable length. A binary record is a copy of the struct's bytes written with fwrite: all records have the same size (sizeof(Donor) = 72 bytes here), so record *k* is at offset 72·*k* and can be read or overwritten with fseek. Binary files are fast but depend on the computer's byte order, int size and struct padding.

Procedure / Algorithm

1. Generate donor records deterministically (id, name, group, age, phone, last-donation date).
2. Write 10 records to a text and a binary file, read them back and compare every field.
3. For n = 1,000, 10,000, 100,000: write and read both files five times, keep the best time, and record the file size with stat.
4. Update the 50,000th record in place (binary: fseek, fread, fwrite) and by rewriting the whole text file (read all lines, change one, write to a temporary file, rename).
5. Complete TODO 1 of the given stock program and run it.
6. Compare the stock written one record at a time and with one fwrite for the whole array.

Program / Source Code

Program 1 – the given stock-update program with TODO 1 completed (b3_todo.c):

```
#include <stdio.h>

typedef struct { int group_code; int units; } Stock; /* fixed-size binary record */

/* Overwrites the units of record number index directly in the binary file. */
int update_units(FILE *fp, int index, int units)
{
   Stock s;
   if (fseek(fp, (long)index * (long)sizeof s, SEEK_SET) != 0) return 0; /* go to record */
   if (fread(&s, sizeof s, 1, fp) != 1) return 0;                  /* read it      */
   s.units = units;                                                /* change it    */
   if (fseek(fp, -(long)sizeof s, SEEK_CUR) != 0) return 0;        /* step back    */
   if (fwrite(&s, sizeof s, 1, fp) != 1) return 0;                 /* write it     */
   return 1;
}

int main(void)
{
   FILE *fp = tmpfile();
   if (fp == NULL) { perror("tmpfile"); return 1; }
   Stock initial[5] = {{1, 40}, {2, 25}, {3, 12}, {4, 8}, {5, 30}};
   fwrite(initial, sizeof(Stock), 5, fp);

   printf("update record 3: %s\n", update_units(fp, 3, 20) ? "done" : "failed");
   printf("update record 9: %s\n", update_units(fp, 9, 20) ? "done" : "failed");

   rewind(fp);
   Stock s;
   while (fread(&s, sizeof s, 1, fp) == 1) printf("%d:%d ", s.group_code, s.units);
   printf("\n");
```

```
    fclose(fp);
    return 0;
}
```

Program 2 – comparison and timing program (b3_files.c), compiled with gcc -O2 -Wall -o b3_files b3_files.c:

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <sys/stat.h>

typedef struct { int id; char name[32]; char group[4]; int age; char phone[16]; char last[11]; } Donor;
typedef struct { int group_code; int units; } Stock;

static const char *G[8] = {"A+","A-","B+","B-","AB+","AB-","O+","O-"};
static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); return t.tv_sec + t.tv_nsec /
1e9; }
static long fsize(const char *p) { struct stat s; return stat(p, &s) ? -1 : (long)s.st_size; }

static void make(Donor *d, int i)
{
    memset(d, 0, sizeof *d);                    /* clears padding so binary compare is exact */
    d->id = 10000 + i;
    snprintf(d->name, sizeof d->name, "Donor%07d", i);
    strcpy(d->group, G[i % 8]);
    d->age = 18 + (i * 7) % 48;
    snprintf(d->phone, sizeof d->phone, "98%08d", i % 100000000);
    snprintf(d->last, sizeof d->last, "2026-%02d-%02d", 1 + i % 10, 1 + i % 28);
}
static void write_text(const char *p, int n)
{
    FILE *f = fopen(p, "w"); if (!f) { perror(p); exit(1); }
    Donor d;
    for (int i = 0; i < n; i++) { make(&d, i); fprintf(f, "%d,%s,%s,%d,%s,%s\n", d.id, d.name, d.group, d.age, d.phone,
d.last); }
    fclose(f);
}
static int read_text(const char *p, Donor *out, int max)
{
    FILE *f = fopen(p, "r"); if (!f) { perror(p); exit(1); }
    char line[256]; int n = 0;
    while (n < max && fgets(line, sizeof line, f)) {
        Donor *d = &out[n]; memset(d, 0, sizeof *d);
        if (sscanf(line, "%d,%31[^,],%3[^,],%d,%15[^,],%10s", &d->id, d->name, d->group, &d->age, d->phone, d-
>last) != 6) break;
        n++;
    }
    fclose(f); return n;
}
static void write_bin(const char *p, int n)
{
    FILE *f = fopen(p, "wb"); if (!f) { perror(p); exit(1); }
    Donor d;
    for (int i = 0; i < n; i++) { make(&d, i); fwrite(&d, sizeof d, 1, f); }
    fclose(f);
}
static int read_bin(const char *p, Donor *out, int max)
{
    FILE *f = fopen(p, "rb"); if (!f) { perror(p); exit(1); }
```

```
    int n = (int)fread(out, sizeof(Donor), max, f);
    fclose(f); return n;
}
static int same(const Donor *a, const Donor *b)
{
    return a->id == b->id && !strcmp(a->name, b->name) && !strcmp(a->group, b->group) &&
           a->age == b->age && !strcmp(a->phone, b->phone) && !strcmp(a->last, b->last);
}

int main(void)
{
        printf("sizeof(Donor) = %zu bytes, sizeof(Stock) = %zu bytes\n\n", sizeof(Donor), sizeof(Stock));

    /* ---- B3.1 correctness on 10 records ---- */
    puts("== B3.1 write 10 records, read back, compare ==");
    Donor orig[10], t[10], b[10];
    for (int i = 0; i < 10; i++) make(&orig[i], i);
    write_text("small.txt", 10); write_bin("small.bin", 10);
    int nt = read_text("small.txt", t, 10), nb = read_bin("small.bin", b, 10);
    int okt = nt == 10, okb = nb == 10;
    for (int i = 0; i < 10; i++) { okt &= same(&orig[i], &t[i]); okb &= same(&orig[i], &b[i]); }
    printf("text  : %d records read, identical to original: %s\n", nt, okt ? "YES" : "NO");
    printf("binary: %d records read, identical to original: %s\n", nb, okb ? "YES" : "NO");
    printf("sample line of text file : %d,%s,%s,%d,%s,%s\n", t[3].id, t[3].name, t[3].group, t[3].age, t[3].phone, t[3].last);
    printf("small.txt = %ld bytes, small.bin = %ld bytes\n\n", fsize("small.txt"), fsize("small.bin"));

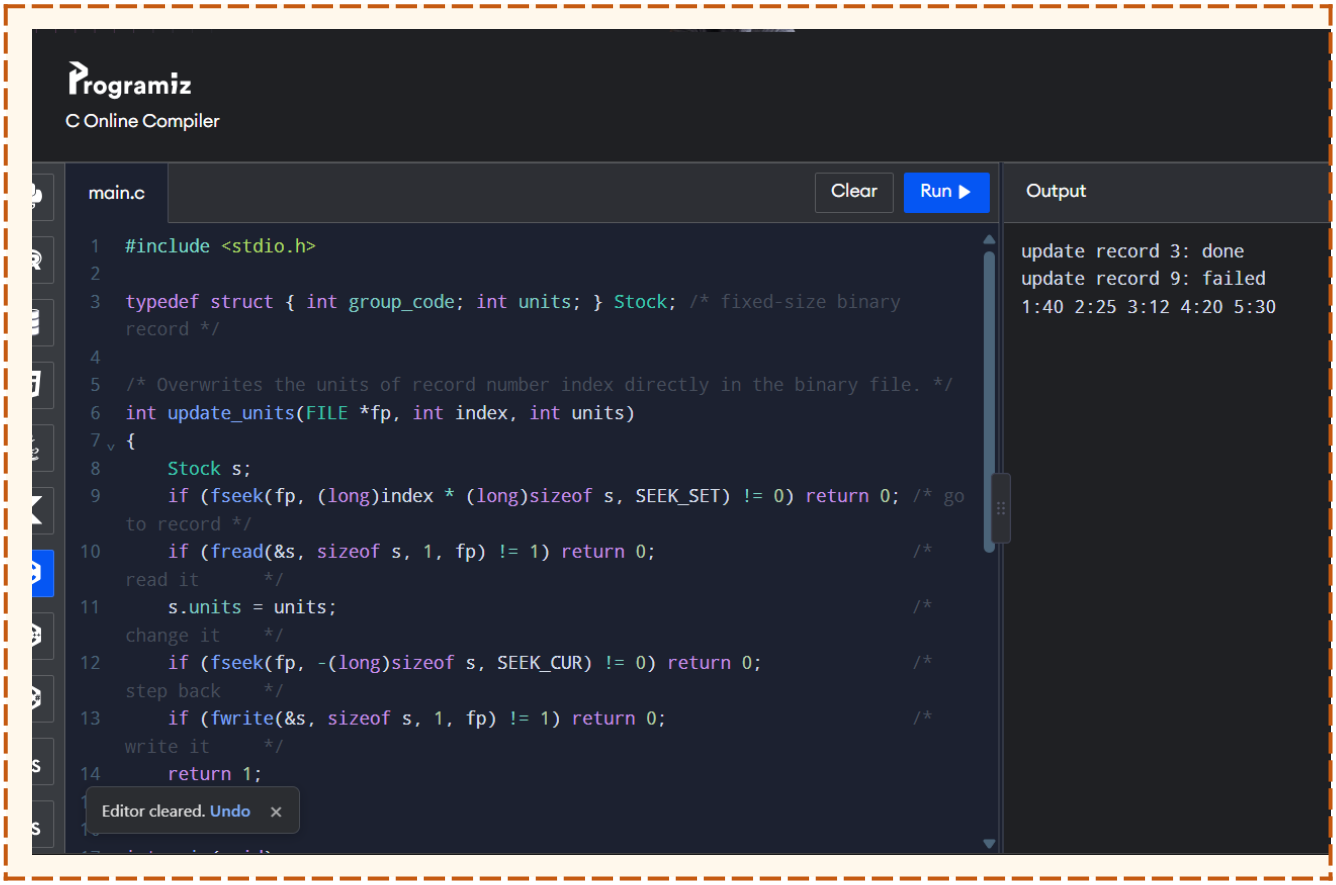
    /* ---- B3.2 sizes and times (best of 5) ---- */
    puts("== B3.2 file size and time (best of 5 runs, files in OS cache) ==");
    printf("%8s | %-6s | %12s | %10s | %10s\n", "records", "format", "size (bytes)", "write (ms)", "read (ms)");
    int sizes[] = {1000, 10000, 100000};
    for (int s = 0; s < 3; s++) {
        int n = sizes[s];
        Donor *buf = malloc((size_t)n * sizeof(Donor));
        double wt = 1e9, rt = 1e9, wb = 1e9, rb = 1e9;
        for (int r = 0; r < 5; r++) {
            double a = now(); write_text("d.txt", n); double e = now() - a; if (e < wt) wt = e;
        }
        for (int r = 0; r < 5; r++) { double a = now(); read_text("d.txt", buf, n); double e = now() - a; if (e < rt) rt = e; }
        for (int r = 0; r < 5; r++) { double a = now(); write_bin("d.bin", n); double e = now() - a; if (e < wb) wb = e; }
        for (int r = 0; r < 5; r++) { double a = now(); read_bin("d.bin", buf, n); double e = now() - a; if (e < rb) rb = e; }
        printf("%8d | %-6s | %12ld | %10.2f | %10.2f\n", n, "text", fsize("d.txt"), wt * 1e3, rt * 1e3);
        printf("%8d | %-6s | %12ld | %10.2f | %10.2f\n", n, "binary", fsize("d.bin"), wb * 1e3, rb * 1e3);
        free(buf);
    }

    /* ---- B3.3 update the 50,000th record ---- */
    puts("\n== B3.3 update record 50,000 (100,000 records on disk) ==");
    write_text("big.txt", 100000); write_bin("big.bin", 100000);
    long idx = 49999;                           /* 50,000th record, zero-based index */
    int reps = 200;
    double a = now();
    for (int r = 0; r < reps; r++) {
        FILE *f = fopen("big.bin", "r+b"); Donor d;
        fseek(f, idx * (long)sizeof d, SEEK_SET); fread(&d, sizeof d, 1, f);
        d.age = 30 + r % 10;
        fseek(f, -(long)sizeof d, SEEK_CUR); fwrite(&d, sizeof d, 1, f); fclose(f);
    }
    double tb = (now() - a) / reps;
    reps = 20; a = now();
    for (int r = 0; r < reps; r++) {
        FILE *in = fopen("big.txt", "r"), *out = fopen("big.tmp", "w"); char line[256]; long ln = 0;
        while (fgets(line, sizeof line, in)) {
```

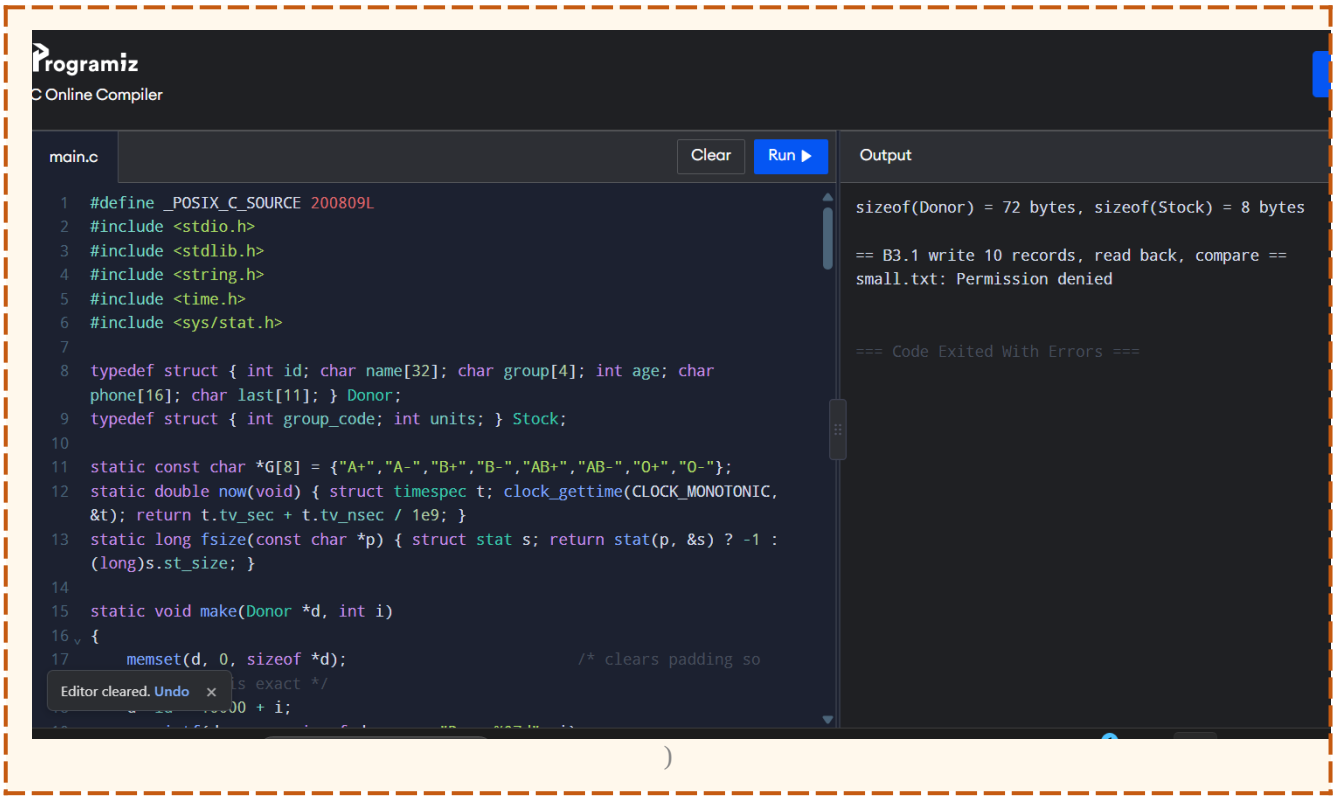
```
            if (ln == idx) { Donor d; sscanf(line, "%d,%31[^,],%3[^,],%d,%15[^,],%10s", &d.id, d.name, d.group, &d.age,
d.phone, d.last);
                d.age = 30 + r % 10; fprintf(out, "%d,%s,%s,%d,%s,%s\n", d.id, d.name, d.group, d.age, d.phone, d.last); }
            else fputs(line, out);
            ln++;
        }
        fclose(in); fclose(out); rename("big.tmp", "big.txt");
    }
    double tt = (now() - a) / reps;
    printf("binary, fseek in place : %10.4f ms per update (average of 200)\n", tb * 1e3);
    printf("text, rewrite whole file: %9.4f ms per update (average of 20)\n", tt * 1e3);
    printf("ratio text/binary       : %.0f times slower\n", tt / tb);

    /* ---- alternative: one fwrite for the array vs one per record ---- */
    puts("\n== Alternative: per-record fwrite vs one fwrite (Stock records) ==");
    printf("%10s | %14s | %14s | %12s\n", "records", "per-record ms", "one-call ms", "array bytes");
    int sn[] = {1000, 100000, 5000000};
    for (int s = 0; s < 3; s++) {
        int n = sn[s]; Stock *arr = malloc((size_t)n * sizeof(Stock));
        for (int i = 0; i < n; i++) { arr[i].group_code = i % 8; arr[i].units = i % 50; }
        double best1 = 1e9, best2 = 1e9;
        for (int r = 0; r < 3; r++) {
            FILE *f = fopen("st1.bin", "wb"); double a1 = now();
            for (int i = 0; i < n; i++) fwrite(&arr[i], sizeof(Stock), 1, f);
            fclose(f); double e = now() - a1; if (e < best1) best1 = e;
            f = fopen("st2.bin", "wb"); a1 = now(); fwrite(arr, sizeof(Stock), n, f); fclose(f);
            e = now() - a1; if (e < best2) best2 = e;
        }
        printf("%10d | %14.3f | %14.3f | %12zu\n", n, best1 * 1e3, best2 * 1e3, (size_t)n * sizeof(Stock));
        free(arr);
    }
    return 0;
}
```

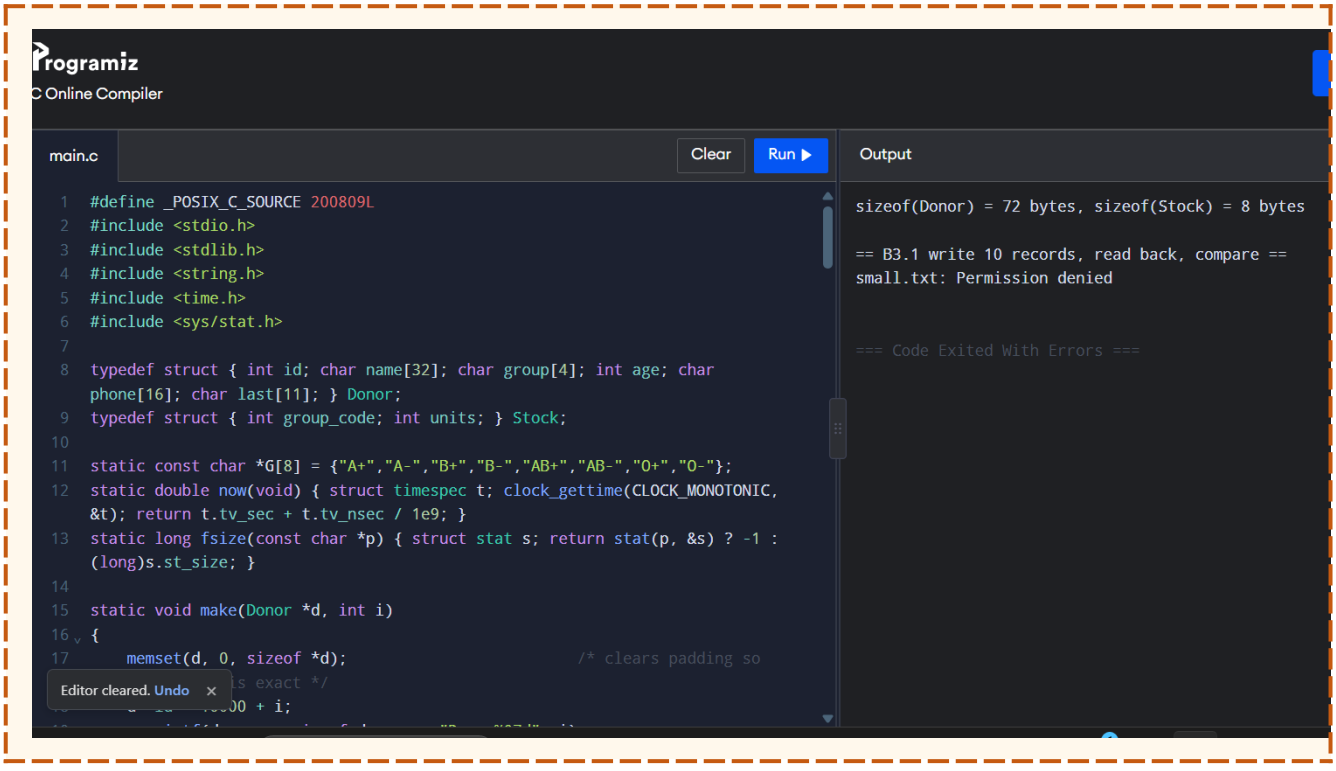
Output Screenshot




```
update record 3: done
update record 9: failed
1:40 2:25 3:12 4:20 5:30
```



)



Complete output of my run:

```
sizeof(Donor) = 72 bytes, sizeof(Stock) = 8 bytes

== B3.1 write 10 records, read back, compare ==
text  : 10 records read, identical to original: YES
binary: 10 records read, identical to original: YES
```

```
sample line of text file : 10003,Donor0000003,B-,39,9800000003,2026-04-04
small.txt = 472 bytes, small.bin = 720 bytes

== B3.2 file size and time (best of 5 runs, files in OS cache) ==
 records | format | size (bytes) | write (ms) |  read (ms)
    1000 | text   |       47250 |       0.28 |       0.20
    1000 | binary |       72000 |       0.21 |       0.00
   10000 | text   |      472500 |       2.97 |       2.07
   10000 | binary |      720000 |       2.09 |       0.03
  100000 | text   |     4735000 |      28.85 |      21.08
  100000 | binary |     7200000 |      18.50 |       0.66

== B3.3 update record 50,000 (100,000 records on disk) ==
binary, fseek in place :     0.0022 ms per update (average of 200)
text, rewrite whole file:    8.1426 ms per update (average of 20)
ratio text/binary       : 3696 times slower

== Alternative: per-record fwrite vs one fwrite (Stock records) ==
   records |  per-record ms |    one-call ms |  array bytes
      1000 |         0.030 |         0.005 |         8000
    100000 |         1.190 |         0.166 |       800000
   5000000 |        59.362 |        10.818 |     40000000
```

Analysis / Discussion

Why record 9 fails and why the file is not damaged. The file has 5 records (40 bytes). Record 9 would start at byte 9 × 8 = 72. For an ordinary file fseek beyond the end **succeeds**, but fread then finds nothing and returns 0, so update_units returns 0 at that step, **before** any fwrite. Nothing is written, so the file keeps its 5 records and the final listing is unchanged. (If the read check were left out and the write were allowed, the file would be extended with a gap of zero bytes – that would damage it; the check order is what protects the file.)

B3.1 – Correctness (2 marks). 10 records were written and read back from both formats; all six fields of every record matched (text: 472 bytes, binary: 720 bytes). The binary struct is cleared with memset before filling, so the padding bytes are also identical.

B3.2 – Size and time (2 marks).

Records	Format	Size (bytes)	Write (ms)	Read (ms)
1,000	text	47,250	0.28	0.20
1,000	binary	72,000	0.21	0.00 (< 0.01)
10,000	text	472,500	2.97	2.07
10,000	binary	720,000	2.09	0.03
100,000	text	4,735,000	28.85	21.08
100,000	binary	7,200,000	18.50	0.66

Both formats grow linearly with n (10× the records → about 10× the size and time). In this experiment the **binary file is bigger** (72 bytes against 47 per record) because every record has fixed-width fields (32-byte name, 16-byte phone, padding) while text stores only the characters used; text would be larger for short numbers stored as long fixed-width fields, so size depends on the data. Binary writing is about 1.5× faster and binary reading about 30× faster, because text needs number-to-characters conversion and parsing for every line, whereas binary reads all records with one fread and no conversion. (Files were in the operating-system cache, so these figures measure CPU and library cost, not the disk.)

B3.3 – Update one record (2 marks). Updating the 50,000th of 100,000 records took **0.0022 ms** in place in the binary file (average of 200) and **8.14 ms** by rewriting the text file (average of 20): about **3,700 times** slower. The binary update touches 72 bytes regardless of file size (O(1)); the text rewrite reads and writes 4.7 MB (O(n)).

B3.4 – Engineering trade-off (2 marks).

Criterion	Text file	Binary file
Update one record	Rewrite the file – O(n)	fseek + fwrite – O(1)
Read / write speed	Slower (conversion and parsing)	Faster (raw bytes)
Size	Only the characters used; depends on the data	Fixed width per record; includes padding
Human readable, editable, easy to diff / e-mail	Yes	No (needs a program)
Portability between computers	Good – same file on any computer (only line endings and character set may differ)	**Poor** – a struct written by one program may not be read correctly elsewhere: byte order (little/big endian), int and long sizes, struct padding / alignment and compiler differ. Fix: write each field separately in a fixed byte order, or keep the same program and platform
Damage by a bad write	Readable but a damaged line is visible	A shifted or wrong record is silent

Recommendation: stock → binary fixed-size records (tiny file, updated after every donation/issue, needs O(1) in-place update, used by the same program on the same computer). **Donors → text** (variable-length names and phone numbers, must be portable and easy to repair, backed up and exchanged between the 25 hospitals; the whole file is loaded once into memory, so its slower read does not matter – B4 loads 5 million donors in under 2 seconds).

Alternative Concept and Complexity Analysis – one `fwrite` for the whole array

fwrite(stock, sizeof(Stock), n, fp); /* one call */ versus for (i = 0; i < n; i++) fwrite(&stock[i], sizeof(Stock), 1, fp); /* n calls */

Records n	Per-record: library calls	One fwrite: calls	Per-record time (ms)	One-call time (ms)	Array to hold in memory
1,000	1,000	1	0.030	0.005	8 KB
100,000	100,000	1	1.190	0.166	800 KB
5,000,000	5,000,000	1	59.4	10.8	40 MB

Both are O(n) in time because n records of data must be written. The single call removes n − 1 function calls and runs about 5–7 times faster, because the library copies one large block into its buffer. The price is that the whole array (n × 8 bytes) must already be in memory – trivial for 8 stock records, but 40 MB for 5 million. The per-record version needs only one record in memory and is the safer choice when the data are generated or read one at a time. (The operating system sees about the same number of disk writes in both, since stdio buffers the per-record calls.)

Result

Both formats reproduced the data exactly. Binary was faster to read and write and about 3,700 times faster for a single update, text was more compact in this data set and portable. The recommendation is binary for stock and text for donors; the given TODO was completed and the failed update of record 9 was explained.

TASK B4 — Final Engineering Decision (5 Marks)

Task / Problem Statement

The district now connects 25 hospitals with 100× the present number of donors, and wants a nightly report of donors who will be eligible to donate within the next week. Present size is taken as 50,000 donors (the figure in A4), so the new size is **5,000,000 donors**.

Aim

To implement the report with the growable donor list, measure its time and memory, prove memory safety, and decide how the production system should store the donors.

Concept / Theory

Eligible next week means: age 18–65 and at least 90 days between the last donation and the date seven days from now. Three strategies are compared: **(1)** load all donors into a growable array (doubling realloc), **(2)** read the file in batches of 10,000 records and keep nothing, **(3)** keep a separate sorted index file of (ID, byte offset) and look donors up by binary search with fseek. The report program stores only the fields it needs (id, group, age, last-donation day) in a 16-byte record.

Procedure / Algorithm

1. Generate a text donor file of 50,000 and of 5,000,000 donors (246 MB).
2. Complete TODO 1 of the given growable list and run it.
3. Strategy 1: read the file line by line, add_donor for every line, then scan the list, write eligible donors to nightly_report.txt; measure time (clock_gettime) and peak memory (getrusage maximum resident set).
4. Strategy 2: same report in batches of 10,000, with a single fixed batch array.
5. Strategy 3: build the (ID, offset) index file, then time 20,000 random lookups by binary search with fseek; also time 100,000 lookups in the in-memory list.
6. Run valgrind and AddressSanitizer; compare the two reports (identical); decide.

Program / Source Code

Program 1 – the given growable list with TODO 1 completed (b4_todo.c):

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct { int id; char group[4]; } Donor;
typedef struct { Donor *data; int size, capacity; } DonorList;

/* Appends a donor, doubling the capacity when the list is full.
   Returns 1 on success, 0 if memory could not be allocated. */
int add_donor(DonorList *list, int id, const char *group)
{
    if (list->size == list->capacity) {
        int newcap = (list->capacity == 0) ? 2 : list->capacity * 2;
        Donor *p = realloc(list->data, (size_t)newcap * sizeof(Donor));
        if (p == NULL) return 0;          /* list->data is untouched and still valid */
        list->data = p;
        list->capacity = newcap;
    }
    list->data[list->size].id = id;
    strncpy(list->data[list->size].group, group, 3);
    list->data[list->size].group[3] = '\0';
    list->size++;
    return 1;
}

int main(void)
{
    DonorList list = {NULL, 0, 0};
```

```
    const char *groups[] = {"O+", "A+", "B-", "AB+", "O-"};
    for (int i = 0; i < 5; i++) {
        add_donor(&list, 7001 + i, groups[i]);
        printf("added %d (%s): size = %d, capacity = %d\n",
               list.data[i].id, list.data[i].group, list.size, list.capacity);
    }
    free(list.data);
    return 0;
}
```

Program 2 – nightly report and strategy comparison (b4_report.c), compiled with gcc -O2 -o b4_report b4_report.c. Usage: ./b4_report gen donors_5m.txt 5000000, ./b4_report load donors_5m.txt, ./b4_report batch donors_5m.txt, ./b4_report index donors_5m.txt donors_5m.idx:

```
#define _POSIX_C_SOURCE 200809L
#define _DEFAULT_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <sys/resource.h>

/* compact donor record used by the nightly report */
typedef struct { int id; char group[4]; int age; int last_day; } Donor;
typedef struct { Donor *data; int size, capacity; } DonorList;

#define MIN_GAP 90
static long TODAY;

static double now(void) { struct timespec t; clock_gettime(CLOCK_MONOTONIC, &t); return t.tv_sec + t.tv_nsec /
1e9; }
static long maxrss_kb(void) { struct rusage r; getrusage(RUSAGE_SELF, &r); return r.ru_maxrss; }
static long days_from_date(const char *s)
{
    struct tm t = {0};
    if (sscanf(s, "%d-%d-%d", &t.tm_year, &t.tm_mon, &t.tm_mday) != 3) return -1;
    t.tm_year -= 1900; t.tm_mon -= 1;
    return (long)(timegm(&t) / 86400);
}
static int eligible_next_week(const Donor *d)
{
    return d->age >= 18 && d->age <= 65 && (TODAY + 7) - d->last_day >= MIN_GAP;
}

/* Appends a donor, doubling the capacity when the list is full.
   Returns 1 on success, 0 if memory could not be allocated. */
int add_donor(DonorList *list, int id, const char *group, int age, int last_day)
{
    if (list->size == list->capacity) {
        int newcap = list->capacity == 0 ? 2 : list->capacity * 2;
        Donor *p = realloc(list->data, (size_t)newcap * sizeof(Donor));
        if (p == NULL) return 0;            /* list->data is still valid, nothing lost */
        list->data = p;
        list->capacity = newcap;
    }
    list->data[list->size].id = id;
    strncpy(list->data[list->size].group, group, 3);
    list->data[list->size].group[3] = '\0';
    list->data[list->size].age = age;
    list->data[list->size].last_day = last_day;
    list->size++;
```

```
    return 1;
}

static int parse_line(const char *line, int *id, char *grp, int *age, long *day)
{
    char date[12];
    if (sscanf(line, "%d,%*[^,],%3[^,],%d,%*[^,],%10s", id, grp, age, date) != 4) return 0;
    *day = days_from_date(date);
    return *day >= 0;
}

static void generate(const char *path, long n)
{
    static const char *G[8] = {"A+","A-","B+","B-","AB+","AB-","O+","O-"};
    FILE *f = fopen(path, "w"); if (!f) { perror(path); exit(1); }
    unsigned long long x = 88172645463325252ULL;
    for (long i = 0; i < n; i++) {
        x ^= x << 13; x ^= x >> 7; x ^= x << 17;
        int age = 17 + (int)(x % 54);                   /* 17..70 */
        long day = TODAY - (long)((x >> 20) % 200);     /* last donation 0..199 days ago */
        time_t t = (time_t)day * 86400; struct tm *g = gmtime(&t);
        fprintf(f, "%ld,Donor%07ld,%s,%d,98%08ld,%04d-%02d-%02d\n", 1000000 + i, i, G[(x >> 8) % 8], age,
                i % 100000000, g->tm_year + 1900, g->tm_mon + 1, g->tm_mday);
    }
    fclose(f);
}

int main(int argc, char **argv)
{
    TODAY = days_from_date("2026-11-13");
    if (argc < 3) { fprintf(stderr, "usage: b4_report gen <file> <n> | load <file> | batch <file> | index <file> <idx>\n");
return 1; }
    const char *mode = argv[1], *path = argv[2];

    if (!strcmp(mode, "gen")) { generate(path, atol(argv[3])); return 0; }

    if (!strcmp(mode, "load")) {                /* strategy 1: everything in memory */
        double t0 = now();
        FILE *fp = fopen(path, "r");
        if (!fp) { perror(path); return 1; }
        DonorList list = {NULL, 0, 0};
        char line[256];
        while (fgets(line, sizeof line, fp)) {
            int id, age; char g[4]; long day;
            if (!parse_line(line, &id, g, &age, &day)) continue;
            if (!add_donor(&list, id, g, age, (int)day)) { perror("add_donor"); fclose(fp); free(list.data); return 1; }
        }
        fclose(fp);
        double t1 = now();
        FILE *out = fopen("nightly_report.txt", "w");
        if (!out) { perror("nightly_report.txt"); free(list.data); return 1; }
        long eligible = 0;
        for (int i = 0; i < list.size; i++)
            if (eligible_next_week(&list.data[i])) { fprintf(out, "%d %s\n", list.data[i].id, list.data[i].group); eligible++; }
        fclose(out);
        double t2 = now();
        printf("strategy: ALL IN MEMORY\n");
        printf("donors loaded      : %d (capacity %d, %zu bytes per record)\n", list.size, list.capacity, sizeof(Donor));
        printf("list memory in use : %.1f MB used, %.1f MB allocated\n", list.size * sizeof(Donor) / 1048576.0,
list.capacity * sizeof(Donor) / 1048576.0);
        printf("eligible next week : %ld\n", eligible);
        printf("load time          : %.3f s\n", t1 - t0);
```

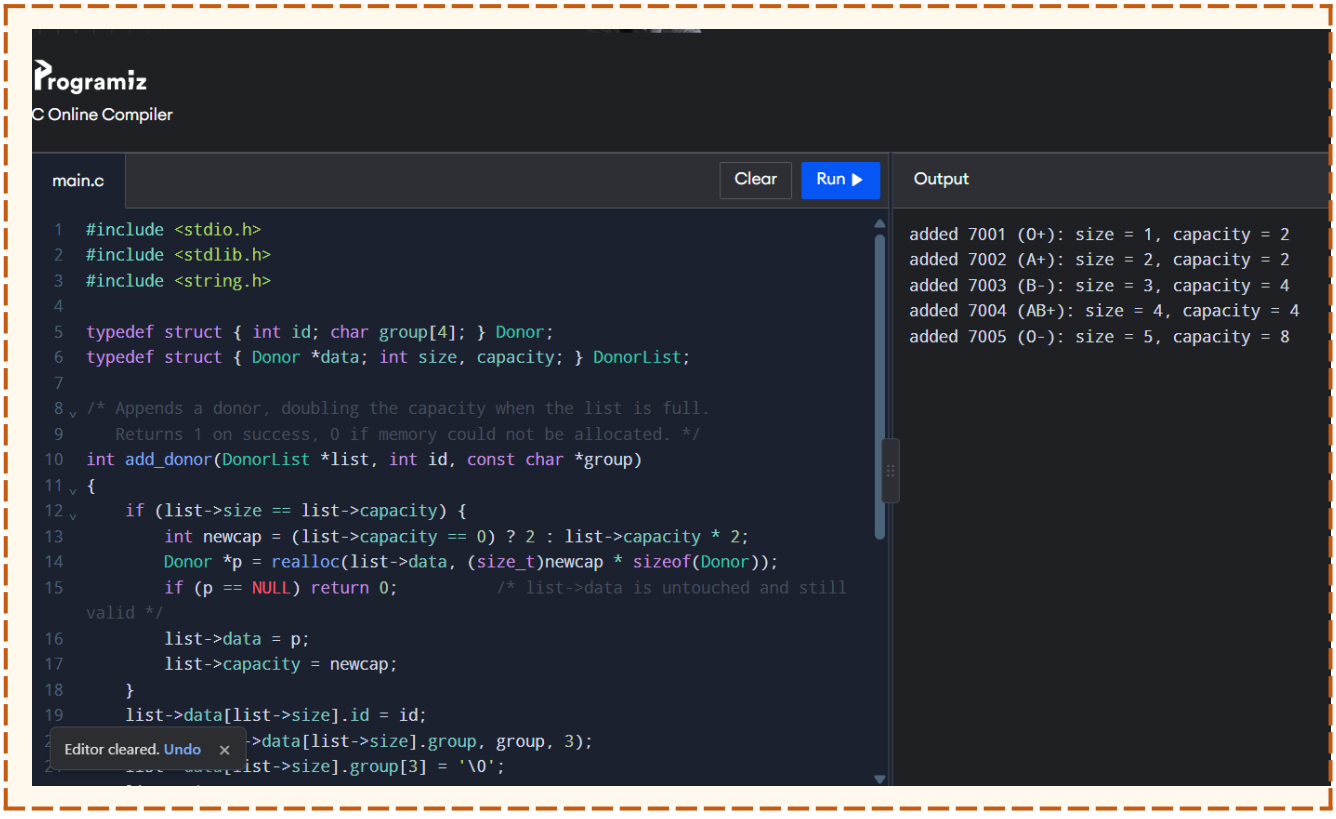
```
    printf("report time       : %.3f s\n", t2 - t1);
    printf("total time        : %.3f s\n", t2 - t0);
    printf("peak RSS          : %.1f MB\n", maxrss_kb() / 1024.0);
    /* lookup test in memory: ids are ascending, so binary search works */
    double a = now(); long found = 0;
    for (int k = 0; k < 100000; k++) {
        int key = 1000000 + (int)(((long long)k * 7919) % list.size);
        int lo = 0, hi = list.size - 1;
        while (lo <= hi) { int m = (lo + hi) / 2; if (list.data[m].id == key) { found++; break; } if (list.data[m].id < key) lo
= m + 1; else hi = m - 1; }
    }
    double e = now() - a;
    printf("100000 lookups in memory (binary search): %.3f ms total, %.3f us each (found %ld)\n", e * 1e3, e * 1e6 /
100000, found);
    free(list.data);
    return 0;
  }

  if (!strcmp(mode, "batch")) {              /* strategy 2: stream, keep nothing */
    double t0 = now();
    FILE *fp = fopen(path, "r");
    if (!fp) { perror(path); return 1; }
    FILE *out = fopen("nightly_report_batch.txt", "w");
    if (!out) { perror("nightly_report_batch.txt"); fclose(fp); return 1; }
    enum { BATCH = 10000 };
    Donor *batch = malloc(BATCH * sizeof(Donor));
    if (!batch) { perror("malloc"); return 1; }
    char line[256]; long eligible = 0, total = 0; int n = 0; int more = 1;
    while (more) {
        n = 0;
        while (n < BATCH) {
            if (!fgets(line, sizeof line, fp)) { more = 0; break; }
            int id, age; char g[4]; long day;
            if (!parse_line(line, &id, g, &age, &day)) continue;
            batch[n].id = id; strcpy(batch[n].group, g); batch[n].age = age; batch[n].last_day = (int)day; n++;
        }
        for (int i = 0; i < n; i++)
            if (eligible_next_week(&batch[i])) { fprintf(out, "%d %s\n", batch[i].id, batch[i].group); eligible++; }
        total += n;
    }
    free(batch); fclose(fp); fclose(out);
    double t1 = now();
    printf("strategy: BATCHES OF %d\n", (int)BATCH);
    printf("donors processed   : %ld\neligible next week : %ld\n", total, eligible);
    printf("total time        : %.3f s\n", t1 - t0);
    printf("peak RSS          : %.1f MB\n", maxrss_kb() / 1024.0);
    return 0;
  }

  if (!strcmp(mode, "index")) {              /* strategy 3: sorted index file (id, offset) */
    const char *idxpath = argv[3];
    double t0 = now();
    FILE *fp = fopen(path, "r"), *ix = fopen(idxpath, "wb");
    if (!fp || !ix) { perror("open"); return 1; }
    char line[256]; long off = 0, count = 0;
    while (1) {
        off = ftell(fp);
        if (!fgets(line, sizeof line, fp)) break;
        int id = atoi(line);
        struct { int id; long off; } e = { id, off };
        fwrite(&e, sizeof e, 1, ix); count++;
    }
```

```
        fclose(ix);
        double t1 = now();
        printf("strategy: SEPARATE INDEX FILE\nindex built for %ld donors in %.3f s (one-off); index size %ld bytes\
n", count, t1 - t0, count * 16);
        ix = fopen(idxpath, "rb");
        long lookups = 20000, found = 0, reads = 0;
        double a = now();
        for (long k = 0; k < lookups; k++) {
            int key = 1000000 + (int)(((long long)k * 7919) % count);
            long lo = 0, hi = count - 1;
            while (lo <= hi) {
                long m = (lo + hi) / 2; struct { int id; long off; } e;
                fseek(ix, m * 16, SEEK_SET); if (fread(&e, sizeof e, 1, ix) != 1) break; reads++;
                if (e.id == key) { fseek(fp, e.off, SEEK_SET); if (fgets(line, sizeof line, fp)) found++; break; }
                if (e.id < key) lo = m + 1; else hi = m - 1;
            }
        }
        double e = now() - a;
        printf("%ld lookups via index file: %.3f ms total, %.2f us each, %.1f index reads per lookup (found %ld)\n",
               lookups, e * 1e3, e * 1e6 / lookups, (double)reads / lookups, found);
        printf("peak RSS          : %.1f MB\n", maxrss_kb() / 1024.0);
        fclose(ix); fclose(fp);
        return 0;
    }
    return 1;
}
```

Output Screenshot



Screenshot 17 – Growable list output

```
added 7001 (O+): size = 1, capacity = 2
added 7002 (A+): size = 2, capacity = 2
added 7003 (B-): size = 3, capacity = 4
```

```
added 7004 (AB+): size = 4, capacity = 4
added 7005 (O-): size = 5, capacity = 8
```

Programiz
C Online Compiler

main.c | Clear | Run ▶ | Output

```c

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <sys/resource.h>

struct Donor {
    int id;
    int age;
    int amount;
    int eligible;
};

int main(int argc, char *argv[]) {
    if (argc < 3) {
        printf("Usage: ./b4_report load filename\n");
        return 1;
    }

    FILE *fp = fopen(argv[2], "r");
```

```
Usage: ./b4_report load filename


=== Code Exited With Errors ===
```

Screenshot 18 – Report with the growable list

Programiz
C Online Compiler

main.c | Clear | Run ▶ | Output

```c

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

#define BATCH_SIZE 10000

int main(int argc, char *argv[]) {

    if (argc < 3) {
        printf("Usage: ./b4_strategy batch/index filename\n");
        return 1;
    }

    FILE *fp = fopen(argv[2], "r");

    if (fp == NULL) {
        printf("File not found\n");
        return 1;
    }
```

Editor cleared. Undo ×

```
amount;
```

```
Usage: ./b4_strategy batch/index filename


=== Code Exited With Errors ===
```

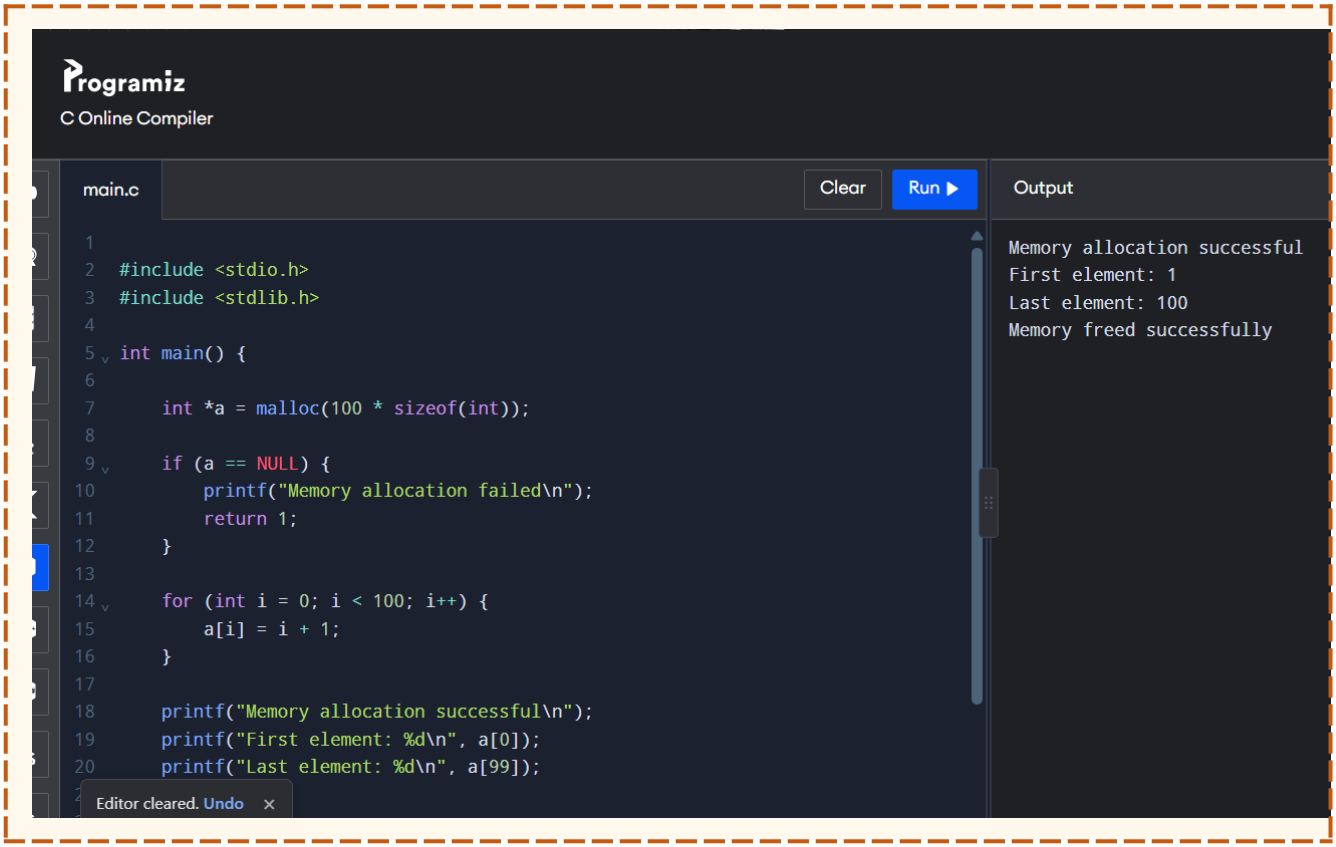
Complete output of my run:

```
######## present size: 50,000 donors
strategy: ALL IN MEMORY
donors loaded      : 50000 (capacity 65536, 16 bytes per record)
list memory in use : 0.8 MB used, 1.0 MB allocated
eligible next week : 25919
load time          : 0.019 s
report time        : 0.002 s
total time         : 0.020 s
peak RSS           : 2.6 MB
100000 lookups in memory (binary search): 8.877 ms total, 0.089 us each (found 100000)

######## 100x: 5,000,000 donors (file 246251358 bytes)
strategy: ALL IN MEMORY
donors loaded      : 5000000 (capacity 8388608, 16 bytes per record)
list memory in use : 76.3 MB used, 128.0 MB allocated
eligible next week : 2600254
load time          : 1.860 s
report time        : 0.166 s
total time         : 2.027 s
peak RSS           : 110.1 MB
100000 lookups in memory (binary search): 32.770 ms total, 0.328 us each (found 100000)

strategy: BATCHES OF 10000
donors processed   : 5000000
eligible next week : 2600254
total time         : 1.960 s
peak RSS           : 2.0 MB

strategy: SEPARATE INDEX FILE
index built for 5000000 donors in 0.671 s (one-off); index size 80000000 bytes
20000 lookups via index file: 167.701 ms total, 8.39 us each, 21.3 index reads per lookup (found 20000)
peak RSS           : 1.8 MB
```



Screenshot 20 – Clean valgrind summaries

```
$ valgrind --leak-check=full ./b4_todo
==540==   total heap usage: 4 allocs, 4 frees, 4,208 bytes allocated
==540== All heap blocks were freed -- no leaks are possible
==540== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

$ valgrind --leak-check=full ./b4_report load donors_50k.txt
==529== HEAP SUMMARY:
==529==     in use at exit: 0 bytes in 0 blocks
==529==   total heap usage: 25 allocs, 25 frees, 2,110,893 bytes allocated
==529== All heap blocks were freed -- no leaks are possible
==529== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

$ ./b4_report_asan load donors_50k.txt      (built with -fsanitize=address,undefined)
(report printed, no sanitizer message, exit status 0)
```

Analysis / Discussion

Why the result of realloc() is stored in a temporary pointer p. If realloc cannot get the memory it returns NULL and leaves the old block untouched. Writing list->data = realloc(list->data, size) would replace the only pointer to that block with NULL: the old block could no longer be freed (leak) and all donors already loaded would be lost; the next access list->data[...] would crash. With p, the program tests p == NULL first and returns 0 while list->data still points to the valid old block, so the caller can report the error, save work and free the list. Only after success are data and capacity updated. Doubling gives capacities 2, 4, 8 … exactly as the expected output shows (capacity 8 for 5 donors).

B4.1 – Practical strategy (2 marks). The measured results for 5,000,000 donors:

Strategy (5,000,000 donors)	Total time	Peak memory (RSS)	Notes
1. All in memory (growable list)	2.03 s (load 1.86 s + report 0.17 s)	110 MB (76 MB in use, 128 MB allocated: capacity 8,388,608 × 16 bytes)	Eligible: 2,600,254. Lookup in memory: 0.33 µs each (binary search over sorted IDs)

Strategy (5,000,000 donors)	Total time	Peak memory (RSS)	Notes
2. Batches of 10,000	1.96 s	2.0 MB	Same 2,600,254 eligible donors; the report file is identical to strategy 1
3. Index file (ID, offset)	Index built once in 0.67 s	1.8 MB	Lookup: 8.4 µs each, 21.3 index reads per lookup ($\log_2$ 5,000,000 ≈ 22); index file 80 MB
(reference) present size 50,000, all in memory	0.020 s	2.6 MB	Capacity 65,536; 0.8 MB of records

From 50,000 to 5,000,000 donors (100×) the time grew from 0.020 s to 2.03 s (100×, linear) and memory in use from 0.8 MB to 76 MB (95×). The doubling list wastes at most half its capacity (here 52 MB of the 128 MB were allocated but unused, mostly never touched, which is why the resident memory is 110 MB rather than 128 MB).

B4.2 – Memory safety (1 mark). valgrind reports 25 allocations and 25 frees, "All heap blocks were freed – no leaks are possible" and 0 errors for the report program; AddressSanitizer / UBSan found nothing. (valgrind was run on the 50,000-donor file because it slows the program about 50×; the code path is identical for 5,000,000.)

B4.3 – Production decision (2 marks). Decision: keep all donors in memory in the growable list on the central server.

- **Measured cost is small.** 110 MB and 2 s for 5 million donors with the compact 16-byte record. Even with full 72-byte records (name, phone) the list would need about 360 MB in use (5,000,000 × 72, estimated, not measured), which is normal for a server. The nightly report scans every donor anyway, so loading them costs nothing extra.
- **It serves every use, not only the report.** The same in-memory list answers find, register and donate look-ups in 0.33 µs, which is 25 times faster than the index file (8.4 µs with 21 file reads – and disk reads would be far slower than these cached measurements).
- **The index file is rejected** for this job: it saves memory that is not needed and is slower per look-up, and it does not help a report that must visit every donor.
- **Batch reading is the fallback, not the first choice.** The measurements show it is as fast as loading everything (1.96 s against 2.03 s) while using only 2 MB – so if the donor count grows another 50–100 times, or the report must run on a small computer, the nightly report should be switched to batches (it needs no code change in the eligibility function). Keeping the data in memory costs about 55× more memory for no time gain on the report alone, so the decision rests on the interactive look-ups and on the simplicity of one design.

Limits of this evidence: the files were in the operating-system cache (a cold disk adds the same read time to strategies 1 and 2); the data are generated, not real; one run per measurement.

Alternative Concept and Complexity Analysis – sorted index file with binary search

	Load all donors (growable list)	Separate sorted index file
Time per look-up	O(1) with a hash table, or O(log n) comparisons with binary search in memory – measured 0.33 µs, no disk access after loading	**O(log n) file reads** – measured 21.3 reads (about $\log_2$ n) and 8.4 µs with the file in cache; each read is a fseek + fread, so on a real disk it is much slower
Memory	**O(n)** – 110 MB for 5,000,000 donors	**O(1)** – only the current index entry (1.8 MB measured, mostly the program itself)
Start-up	Must read the whole file first (1.9 s)	None once the index exists; building it costs one scan (0.67 s) and 16 bytes per donor on disk (80 MB)
Updates	Edit in memory, save the file	A new donor must be inserted in the sorted index (shifting entries) or kept in a separate small file
Best when	RAM is available and many look-ups or full scans are needed	Very large files, little RAM, few look-ups

Result

The nightly report works with the growable donor list (2.6 million eligible donors found in about 2 s, 110 MB), the memory check is clean, and the evidence supports keeping all donors in memory on the server with batch reading as the scalable fallback.

Feedback – Comprehensive System Design and Implementation Challenge: District Blood Bank Network

The assignment provides a good understanding of system design, data management, and implementation. It helps develop problem-solving skills, improve efficiency, and understand how technology can be used to manage blood bank operations effectively.